Pushing Dexterity and Safety for In-Hand Object Pose Reaching That Works Remarkably Well

Ian Chuang[1]

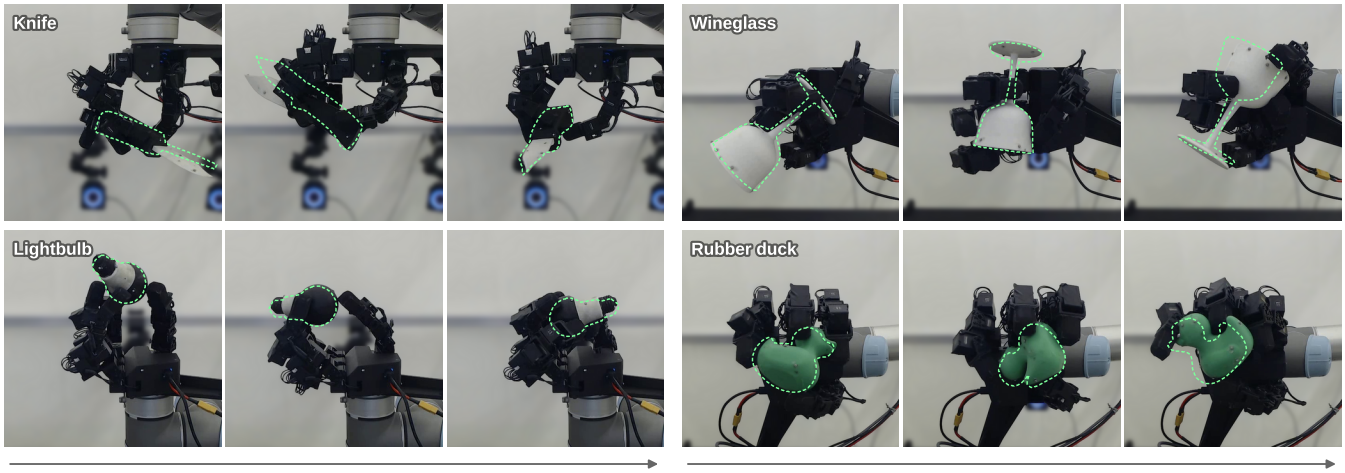


Fig. 1: **Our robot hand reaches diverse in-hand object poses at any wrist orientation.** For each object (one policy per object), we show three consecutive goals; dashed green outlines indicate target poses, and arrows indicate time.

***Abstract*— Dexterous manipulation with multi-fingered hands is the next frontier of robotics, yet learned hand skills remain limited to simple grasps or basic repeated motions, such as rotating an object. As robots attempt more demanding in-hand maneuvers, safety, such as not dropping the object, becomes critical, yet it is rarely measured. We study *in-hand object pose reaching*, a task that demands diverse and difficult grasps: moving an object to any position and orientation relative to the hand, at any wrist orientation, through long sequences of consecutive goals. We present a recipe for training such policies for a LEAP hand in simulation without human data, built on a range of insights, from diverse resets and goals via generated grasp datasets to an adaptive friction curriculum. With careful system identification, the policies transfer zero-shot to the real hand, using either motion capture or a single wrist-mounted depth camera, and prove remarkably capable and robust. On top of the task policy, we add a learned residual safety filter with adjustable conservativeness, letting us tune the balance between performance and safety and greatly reduce object drops. In 13.3 hours of real-robot evaluation over 4,800 unseen goals across 4 objects and diverse wrist orientations, the state-based policy reaches 95% of goals within 2 cm and 0.4 rad, and the filter cuts its drops by 75% at a cost of 3.7 points of success. The policies can also spin a lightbulb held in an arbitrary pose, reach out-of-distribution goals, withstand fast arm motion and perturbations, and, needing only a wrist camera, serve as portable skills such as screwing in a lightbulb. We release the full recipe as a platform for studying performance, reliability, and safety in high-dimensional dexterous systems.**

I. INTRODUCTION

Multi-fingered robot hands promise human-like dexterity, but these high-dimensional, contact-rich systems are very difficult to control and leave little margin for error. Recent advances in reinforcement learning (RL) and imitation learning have produced increasingly capable hand controllers, particularly for in-hand reorientation and grasping [**?**], [**?**], [**?**]. Yet the current landscape of in-hand skills remains narrow. Most systems operate at a single wrist orientation, often palm up, where the object rests stably in the palm, and perform simple, repetitive motions such as object rotation [**?**], [1]. Many of these works also report performance only over short, limited trials and treat reliability and safety as an afterthought, handled implicitly, e.g., through reward shaping, or not at all. Human in-hand dexterity is far broader: a person can move an object into a difficult grasp at almost any position and orientation in the hand, at any wrist orientation, through diverse finger motions, and rarely drop it. Achieving such grasps reliably over long sequences of goals remains an open challenge. As robots move outside the lab, where objects can be fragile or hazardous, reliability (how long the system runs without failing), safety (how often it violates a constraint, such as dropping the object) and robustness (to different wrist orientations, scenes and external disturbances) matter as much as success rate. How far a dexterous system can be pushed while staying safe therefore becomes a central question.

To study this question, we consider *in-hand object pose reaching*: moving an object to any position and orientation relative to the hand, at any wrist orientation, over long sequences of consecutive goals (Fig. 1). Beyond rotation or reorientation [2], [3], [**?**], the task demands precise translation, so the hand must often hold the object at the tip of

[1]University of California, Berkeley [2]University of California, Davis
[†]Corresponding email: tomlin@eecs.berkeley.edu

its fingers, in hard-to-reach positions and without the palm to support it. Its difficulty makes the task both a frontier for sim-to-real dexterity and a natural testbed for pushing performance and safety together.

We share a complete recipe for training a pose-reaching policy in simulation and transferring it zero-shot to a 16-DoF LEAP hand: goals and initial states drawn from large generated grasp datasets, an adaptive friction curriculum, a simple reward of a few dense goal-reaching terms and regularizers, careful system identification of the actuators [**?**], and teacher–student distillation into policies that track the object with either motion capture (the *state policy*) or only a wrist-mounted stereo depth camera (the *depth policy*). To improve the policy's safety, we train a residual safety filter, adapted from ResSafe [**?**], that sits on top of the frozen policy and corrects its actions to keep the hand from dropping the object. It trains in a few hours, and a single dial, the correction weight w, makes it easy to tune how conservative it is at deployment.

We evaluate the system extensively on a real robot hand: 13.3 hours of robot run time over 4,800 unseen, randomly drawn goals, four objects, some of highly nonconvex shape (a knife, a wineglass, a lightbulb and a rubber duck), and six wrist orientations, with and without the safety filter. The state policy reaches 95 % of these goals within 2 cm and 0.4 rad, and the filter cuts its drops by 75 % (from 16 to 4) at a cost of 3.7 points of success. Using only the wrist camera, the depth policy still reaches 72 % of goals (95 and 88 % on the lightbulb and rubber duck), and the filter nearly halves its drops (from 88 to 46). In further experiments, the system reaches out-of-distribution goals, spins a lightbulb held in difficult poses, withstands fast arm motion and physical perturbations, and, since the depth policy needs no external cameras or motion capture, it can easily be deployed with a robot arm as a skill for multi-step tasks, such as screwing in a lightbulb until it lights up, or picking up and reorienting objects.

In summary, our contributions are:

- A sim-to-real recipe for dexterous, robust and reliable *in-hand object pose reaching* with a LEAP hand at arbitrary wrist orientations, using only proprioception and either motion capture or a wrist camera for tracking, with insights and ablations of the main design choices. We will release the simulation training and deployment code as well as the trained checkpoints.
- An adaptation of the learned residual safety filter ResSafe [**?**] to dexterous manipulation, as a simple add-on for learning safety in high-dimensional systems that is quick to train and easy to tune, balancing safety and performance.
- An extensive real-robot evaluation of performance and safety: 13.3 hours of robot run time and 4,800 unseen goals, in which the state policy reaches 95 % of goals and the safety filter cuts its drops by 75 %, together with stress tests that push the policy with difficult goals and disturbances, and demonstrations of the policy as a reusable skill in multi-step tasks, such as screwing in a lightbulb.

II. Related Work

Dexterous in-hand manipulation. Large-scale RL in simulation with domain randomization produced in-hand cube reorientation on a Shadow hand [**?**], [**?**]. Later work rotated objects about arbitrary axes from proprioception [**?**], [1], reoriented novel objects from depth [2], and reached goal orientations and used tools [3], [**?**]. Most of these systems control orientation only, often palm up or as a continuous rotation; we ask for full 6D goals at any wrist orientation. Imitation from human teleoperation [**?**] struggles to produce such dynamic finger gaiting, especially on non-anthropomorphic hands; our policies use no human data. **Goals, resets and sim-to-real transfer.** Like several recent works, we reset episodes to diverse, feasible grasps and draw goals from a generated grasp dataset [**?**], [4], [5], [6], [**?**], and we bridge the sim-to-real gap with an identified actuator model [**?**] and teacher–student distillation to depth [2], [**?**], [**?**].

Learned safety filters. Hamilton–Jacobi (HJ) reachability and control barrier functions (CBFs) give safety filters with guarantees [**?**], [7], [**?**], but they rely on models and computation that do not scale to a 16-DoF hand with intermittent contacts; CBF filters for grasping use simplified contact models [8], [9]. Learned variants approximate the HJ value with a safety Bellman backup [**?**], [10], [11], [**?**] and filter actions with it, either by switching to a fallback policy or through a CBF condition [**?**], which requires an accurate value near the boundary and a run-time search. ResSafe [**?**] instead learns a residual correction on top of a fixed nominal policy [12]. We adapt it to dexterous manipulation, make its conservativeness a run-time input, and compare it with HJ filters.

III. Problem Formulation

We define in-hand pose reaching as the problem of bringing an object to a desired position and orientation relative to the hand. We attach a fixed frame to the object and a fixed frame to the hand (the hand root, below the center of the palm), and express goals in the hand root frame. A goal is either a full $SE(3)$ pose (p^*, R^*) or, for objects that are rotationally symmetric about their z-axis, a goal position p^* and a target axis direction z^*. The axis form leaves the rotation about the object's own axis free, so it can instead be commanded as a spin; this suits objects that look the same at every angle about that axis, whose full pose cannot be recovered from vision alone. With the object at (p, R) in the hand frame, we measure the position and orientation errors as

$$e_p = \|p - p^*\|_2, \quad e_R = \begin{cases} \|\mathrm{Log}(R\,R^{*\top})\|_2 & \text{pose goal,} \\ \arccos\!\left(z^\top z^*\right) & \text{axis goal,} \end{cases} \tag{1}$$

where z is the object's z-axis. For a pose goal, e_R is the geodesic angle on $SO(3)$; for an axis goal, it is the tilt between the object's axis and the target axis. A goal is

reached when e_p and e_R are both below their thresholds. We perform consecutive goal reaching: as soon as one goal is reached, the next one is commanded, without resetting the hand.

For safety, we consider the constraint of not dropping the object, with the safety margin

$$\ell(x) = d_{\max} - \|p_{\text{obj}} - p_{\text{hand}}\|_2, \tag{2}$$

where $p_{\text{obj}} - p_{\text{hand}}$ is the offset from the hand root to the object frame. The safe set is $\{x : \ell(x) \geq 0\}$: when the object is more than $d_{\max}$ from the hand root, $\ell(x) < 0$ and we consider the object dropped. We use $d_{\max} = 0.3\,\text{m}$ for the safety filters and the simulation benchmarks. This is far beyond any goal (all goals lie within about 0.2 m of the hand root), so leaving the safe set means the object has fallen. The objective is to reach diverse and challenging goals while keeping the system safe, i.e., without dropping the object.

IV. Robot System and Simulation

A. Hardware

Our setup is shown in Fig. 2. We use a LEAP hand [13] with a redesigned palm that stabilizes the base joint of the thumb. Its 16 joints are Dynamixel XC330 servos, each running its internal position controller. Only the fingertips of the hand are covered with high-friction 3M grip tape, and the rest of the hand is mostly low-friction plastic. The fingertips themselves are hard 3D-printed plastic. Visual Dexterity [2] found that soft fingertips make manipulation more forgiving, but our recipe performs very well without them. A stereo camera on the wrist (Orbbec Gemini 305) looks at the palm and fingers. We compute depth from its stereo images with Fast FoundationStereo [14] at 30 Hz, which is much cleaner and less noisy than the depth stream the camera itself provides, and downsample it to 160×120 for the policy. The hand is mounted on a UR5 arm, which sets and measures the orientation of the hand, and thus the direction of gravity in the hand frame. The pose-reaching policy controls only the hand, not the arm. A motion-capture system tracks marker-equipped copies of the objects and the hand, both as the object-pose input of the state policy and as ground truth for evaluation. The policy runs at 20 Hz and sends joint position targets to the servos. Appendix A gives the full description of our hardware setup.

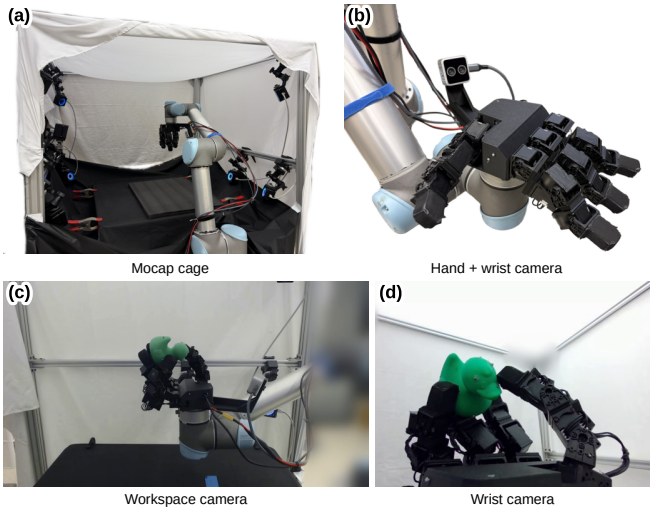


Fig. 2: **Hardware setup.** (a) The UR5 arm with the LEAP hand inside the motion-capture cage. (b) Close-up of the hand and the wrist-mounted stereo camera. (c) Workspace camera view. (d) Wrist camera view.

B. System Identification

Actuator model. Sim-to-real works commonly simulate each position-controlled joint as a simple PD actuator, which turns the position error directly into a torque, $\tau = k_p(q^* - q) - k_d\dot{q}$. Instead, we model the servo itself with BAM [**?**], [15], a voltage-driven DC-motor model with a detailed friction model: we record our own data on a single-servo pendulum test bench and fit the model's parameters to it, and use these parameters for all 16 actuators of the hand (Fig. 15).

Latency calibration. We play the same joint trajectory on the real hand and in simulation, and the time shifts that best align the two give the delays we use in simulation for the joint readings, the position commands and the camera. With the identified servo model and delays, the simulated joint positions and depth images closely follow the real ones (Fig. 16). Appendix B gives the details of both steps.

C. Simulation Environment

We build the simulation in mjlab [16], which runs MuJoCo Warp [**?**], a GPU-accelerated version of MuJoCo [**?**]. The arm is not simulated: the hand is fixed in free space, at a random wrist orientation that changes every episode. We hand-tune our own LEAP hand model, adapted from [**?**], [**?**], [13], adjusting the masses, inertias and collision geometry. Its joints are driven by the identified servo model and delays, and the wrist depth camera is rendered with the calibrated intrinsics and extrinsics of the real camera. Physics runs at 120 Hz and the policy at 20 Hz. For collisions, we decompose each object (Table VI, Fig. 3) into convex parts with CoACD [**?**]. Appendix C gives the full description and the simulator settings. Below we describe the action space, the observations and the domain randomization.

1) Action Space: The action space is inspired by Hora [**?**] and TeleDexter [6]. The policy outputs $a_t \in \mathbb{R}^{16}$, and each control step computes

$$\begin{aligned} s_t &= \alpha\,\text{clip}(a_t, -1, 1) + (1-\alpha)\,s_{t-1}, \\ \Delta_t &= \kappa\,\text{sign}(s_t)\,\max(|s_t| - \tau, 0), \\ \tilde{q}_t &= \text{clip}(q_{t-1}^{\text{cmd}} + \Delta_t,\ q_t - 0.3,\ q_t + 0.3), \\ q_t^{\text{cmd}} &= \text{clip}(\tilde{q}_t,\ q_{\min},\ q_{\max}), \end{aligned} \tag{3}$$

with $\alpha = 0.7$, $\tau = 0.1$ and $\kappa = 0.1$ rad, where q_t is the measured joint position and q^{cmd} the position target sent to the servos, initialized to q_t after a reset. Line by line: the action is clipped to $[-1, 1]$ and smoothed by an exponential moving average s_t. A deadzone, as in TeleDexter, turns s_t into the target increment Δ_t: components with $|s_t| \leq \tau$ give



Fig. 3: **Objects used in our experiments.** For each object we show its simulation mesh, its collision meshes from convex decomposition, the real object, and the real object with motion-capture markers.

no increment, so that small outputs keep the hand still instead of drifting. The increment is added to the previous target, which is kept within 0.3 rad of the measured position so that q^{cmd} cannot drift far from the actual joint, e.g., while a finger is blocked by the object. Finally, the target is clipped to the joint limits. Each joint therefore moves its target by at most 0.09 rad per step (1.8 rad/s). We found the clipping to $[-1, 1]$ together with the small scale of 0.1 rad to be essential to keep the hand from moving too fast, since it limits the maximum speed of the motors. Appendix F further discusses this choice of action space.

2) Observations: The policies draw on the observations of Table IX, grouped by the symbols of Fig. 4. The proprioception o^{prop} is what is also available on the real robot: the gravity direction in the hand frame, a 0.5 s history of joint positions, the residual between the commanded target q^{cmd} and the measured position q, and the last action. x is the object pose (from motion capture on the robot), g the goal, and I the 120×160 wrist depth image. The privileged observations o^{priv} add noise-free states and velocities of the joints, fingertips and object, and fingertip contacts. A few further terms, such as the randomized friction and the success bookkeeping, are given only to the critic (value function) in RL training (Sec. V-A.4), as in asymmetric actor–critic training [**?**]: they help it predict returns, but an actor could exploit them and learn undesired behavior.

To narrow the visual gap, we augment the simulated depth following DextrAH [**?**]: pixel jitter, disparity noise, surface-normal-dependent noise, pixel dropout, random-value pixels and thin stick artifacts. We also randomize the camera pose and field of view, and when training the depth policies we add random floating distractor objects around the hand, so that the depth policy becomes robust to visual distractors (Table VIII, Fig. 18).

3) Domain Randomization: To transfer to the real robot, we randomize the simulation broadly: friction, actuator parameters, masses and inertias, and collision geometry, as well as random pushes on the object, delayed and dropped commands, and noise and latency on the observations the robot provides (Appendix D).

V. Learning Pose Reaching and Safety

Our training pipeline is outlined in Fig. 4. It has two parts. The pose-reaching policy, which we also call the task policy, is trained in four stages (A1–A4), and the safety filter in two (B1–B2) on top of the frozen task policies. Every stage trains one network per object. For each object we obtain two task policies: the *state policy* (π^{s}, A3), which observes the object pose from motion capture, and the *depth policy* (π^{d}, A4), which sees only the wrist depth camera; and a safety filter for each, the state filter (B1) and the depth filter (B2). Sec. V-A describes the task policies, and Sec. V-B the safety filter.

A. Pose-Reaching Policy

We train the pose-reaching policies with RL, using PPO [**?**] in thousands of parallel simulated environments. Below we describe the RL environment built on the simulation of Sec. IV-C: how we set up goals and initial states, the reward, the curricula and the training stages.

1) Goals and Resets from a Grasp Dataset: Rather than sampling object pose goals uniformly over $SE(3)$, most of which no grasp can reach, we generate a grasp dataset for each object and our hand offline, with no human data. LightningGrasp [**?**] produces 100,000 force-balanced fingertip grasps; the RRT formulation of DexterityGen [**?**] expands them into 500,000 grasps with object poses that cover the space above the palm densely; and we remove grasps that have collisions in the object and hand geometry, which leaves 422k–453k grasps per object (Appendix E, Fig. 17). Each grasp stores the 16 joint angles and the object pose in the hand frame. Goals and initial states are drawn from this dataset.

Resets and terminations. In each episode, the hand and object start from a grasp drawn from the dataset, with the hand at a uniformly random wrist orientation. An episode times out at 20 s and terminates early when the object falls more than 0.5 m from the hand (Appendix F.1).

Goal success in training. A goal counts as reached when the object is within three tolerances, and stays there. The position and orientation errors (1) must be below τ_p and τ_R. The object must also move at the goal twist ω^*: we low-pass filter the object's angular velocity ω in the hand frame,

$$\bar{\omega}_t = (1-\alpha_\omega)\,\bar{\omega}_{t-1} + \alpha_\omega\,\omega_t, \qquad e_\omega = \|\bar{\omega}_t - \omega^*\|_2, \quad (4)$$

with $\alpha_\omega = 0.05$, and require the twist error e_ω to be below τ_ω. For the lightbulb, the goal twist is a spin about the goal axis, $\omega^* = \omega^*_z z^*$ with $\omega^*_z \sim \mathcal{U}(-1.5, 1.5)$ rad/s (or zero with probability 0.5); for every other object it is zero. This makes the policy hold the object still, or spin it steadily, at the goal. Finally, the goal must be held: a hold counter, reset to zero for each new goal, goes up by one at every step within all three tolerances and down by one (but not below zero) at every step outside them, and the goal is reached when it

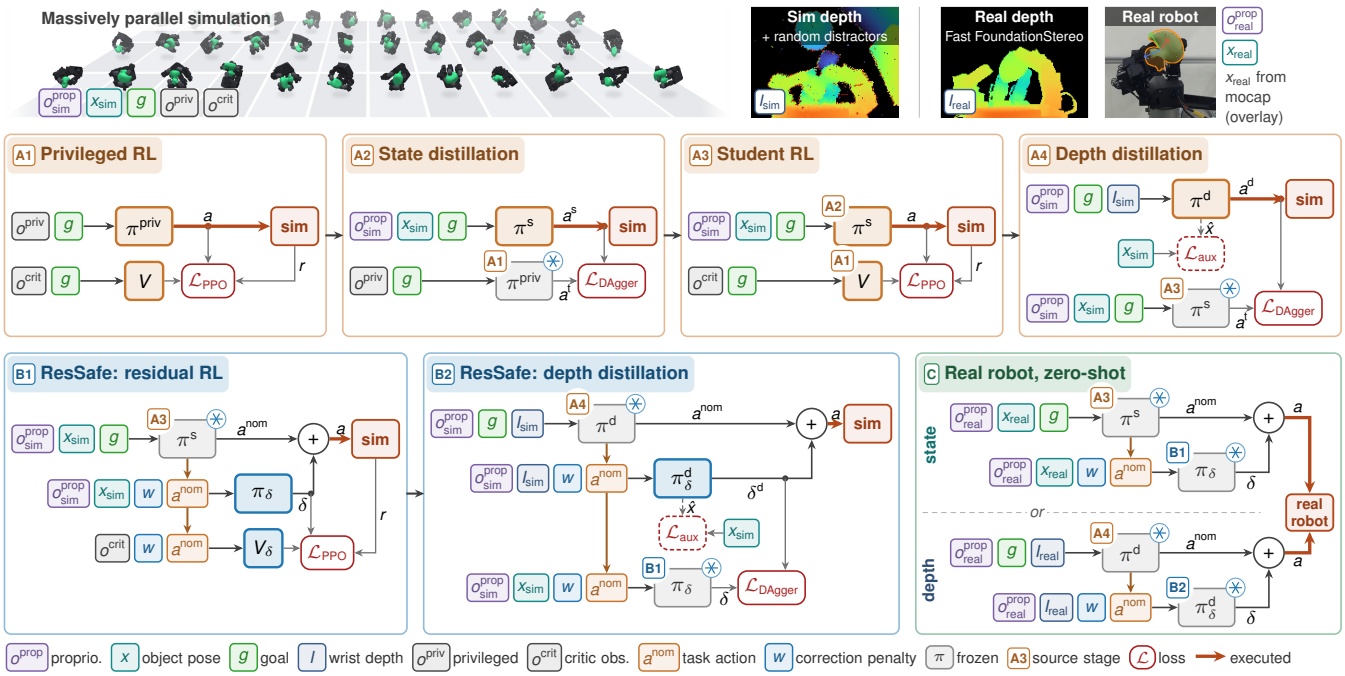


Fig. 4: **Training pipeline.** (a) The task policy: A1 PPO with privileged observations and an asymmetric critic, A2 DAgger into a deployable state student, A3 PPO fine-tuning of that student, and A4 DAgger into a depth student. (b) The safety filter: B1 learns a residual safety filter on top of the frozen A3 policy; B2 distills it into a depth student filter on top of A4. (c) This gives a task policy and a safety filter that use either motion capture or the wrist camera, and both transfer zero-shot to the real robot.

reaches the required hold h_{req}, drawn per goal. A brief slip thus delays success without restarting the count. Table VI lists τ_p, τ_R, τ_ω and the range of h_{req} for each object. A new goal is drawn at every reset and as soon as the current one is reached, so the policy reaches goals consecutively.

2) Reward: We keep the reward simple: dense goal-reaching terms and a few regularizers. We write the goal-reaching terms with potential-based shaping [**?**]: a potential Φ measures progress toward the goal, such as the negative distance to it, and the reward is its change between steps, $\Delta\Phi_t = \Phi_t - \Phi_{t-1}$. Moving toward the goal is rewarded, moving away is penalized by the same amount, and hovering near the goal without reaching it earns nothing. We prefer this to rewards for closeness to the goal [**?**], [**?**], [6], which pay at every step near the goal and so can be exploited by hovering there. The task reward is

$$\begin{aligned} r_t = \Delta t \Big[& 600\,\Delta\Phi_t^p + 25\,\Delta\Phi_t^R + 10\,\Delta\Phi_t^\omega + 35\,\Delta\Phi_t^h \\ & - 50\,\mathbf{1}_{\text{drop}} + 0.01\,(n_t^{\text{tip}} - n_t^{\text{link}} - n_t^{\text{self}}) \\ & - 0.01\,\|a_t - a_{t-1}\|^2 - 0.005\,\|a_t\|^2 \Big], \end{aligned} \tag{5}$$

with the potentials

$$\Phi^p = -\min(e_p,\ 0.1\,\text{m}), \qquad \Phi^R = -e_R, \qquad \Phi^h = h/h_{\text{req}},$$
$$\Phi^\omega = \begin{cases} -\min(e_\omega,\ 1.5) & \text{if } e_p < \tau_p \text{ and } e_R < \tau_R, \\ -1.5 & \text{otherwise.} \end{cases} \tag{6}$$

The position and orientation potentials Φ^p and Φ^R are the negative errors to the goal, the position capped at 10 cm. The twist potential Φ^ω is the negative error to the goal twist, counted only once the pose is within tolerance. The hold potential Φ^h rises from 0 to 1 as the hold counter h fills. The drop penalty $\mathbf{1}_{\text{drop}}$ is applied if the episode terminates with a drop. The contact terms count fingertips touching the object (n^{tip}), other links touching it (n^{link}) and self-contacts (n^{self}), which encourages fingertip grasps and discourages non-fingertip contact and self-collision. The last two terms penalize the action rate and magnitude, for smoother motion. Finally, the sum is scaled by the control period $\Delta t = 0.05$ s. Appendix F explains each term.

3) Friction and Goal Curricula: Curricula speed up learning dramatically. When training the privileged teacher (A1), we use a friction curriculum and a goal curriculum, both adaptive in the style of automatic domain randomization [**?**], [**?**]: each has a progress $p \in [0, 1]$ that steps up when the success rate over the last 500 goals is above a threshold and down when it falls below a slightly lower one. The friction curriculum starts with unrealistically high friction ($\mathcal{U}(2.5, 3.0)$ at the fingertips) and moves it linearly to the final range ($\mathcal{U}(0.3, 1.2)$). It changes the difficulty smoothly without changing the structure of the task. The goal curriculum follows DexterityGen [**?**] and TeleDexter [6]: the next goal is one of the 10 grasps nearest to the current state among a random subset of the dataset, and this subset shrinks from 2048 grasps to 1 as p rises, so goals move from near the current grasp to uniform over the dataset, while the success tolerances τ_p, τ_R and τ_ω tighten and the required

hold h_{req} grows. All later stages run without curricula, at their final settings (low friction and goals uniform over the dataset). We ablate both curricula and also a gravity curriculum, which ramps gravity from zero to its full value [6], [2]: the friction curriculum speeds up learning the most, while the goal and gravity curricula help little on their own (Sec. VI-B.3). Appendix G gives the full description and the schedules.

4) Training Stages: We follow the common recipe of distilling privileged teachers into deployable students. A1 trains a teacher with PPO [**?**], [**?**] on privileged observations $(o^{\text{priv}}, g^{\text{priv}})$ with an asymmetric critic; it is the only stage that uses the friction and goal curricula. We find that A1 learns far better with a privileged actor: an actor with only deployable observations, as in TeleDexter [6] and SimToolReal [3], reaches about half as many goals (Sec. VI-B.3). A2 distils the teacher with online DAgger [**?**] into a student that uses only observations available on the real robot (o^{prop}, x, g), including the object pose from motion capture. A3 fine-tunes this student with PPO in the same environment, which recovers part of the performance lost in the A2 distillation; its critic is copied from A1. A4 distils A3 into a student that replaces the object pose with the wrist depth image, encoded by a CNN with a spatial softmax [**?**]. An auxiliary head and loss predict the object pose from the same features, to help the encoder localize the object [**?**], [**?**], [3]. A4 trains at relaxed success tolerances ($\tau_p = 2\,\text{cm}$, $\tau_R = 0.4\,\text{rad}$), with visual distractors around the hand and a randomized camera pose and field of view (Sec. IV-C.2). The results are the state policy (A3) and the depth policy (A4). On one machine with RTX 5090 GPUs, A1 takes about 15 h on two GPUs (28 h for the wineglass), and A2, A3 and A4 take 0.6, 2.5–2.8 and 10 h on one GPU. Appendix H gives the full description and the hyperparameters.

B. Residual Safety Filter

On top of the frozen task policies, we train a safety filter (Fig. 4b). We follow ResSafe [**?**], which showed that a modular layer can be trained on top of a nominal policy for high-dimensional systems that are hard to model: a learned residual keeps the system alive, i.e., safe, while being encouraged to deviate as little as possible from the nominal policy. This separates learning the task from learning safety. Making the task policy itself safer, e.g., with a larger drop penalty, turns its reward into a multi-objective problem whose trade-off is hard to tune, and every new trade-off would mean retraining the task policy (about 28 h per object). The filter is instead a quick layer on top of the frozen policy (about 4 h of training), and it is easy to tune and deploy.

1) Formulation: Let $a_t^{\text{nom}} = \text{clip}(\pi(\cdot), -1, 1)$ be the action of the frozen task policy. The safety policy π_δ observes a_t^{nom} and outputs a residual δ_t, and the executed action is

$$a_t = \text{clip}(a_t^{\text{nom}} + \delta_t,\ -1,\ 1)\,, \tag{7}$$

which then passes through the same action law (3) as the task policy. The clip keeps the executed action within its bounds whatever the residual. It does not change what the robot does, since the action law clips to $[-1, 1]$ anyway. ResSafe trains π_δ to maximize the discounted return $\mathbb{E}\big[\sum_{t=0}^{T-1} \gamma^t r_t\big]$, where T is the step at which the system leaves the safe set, with a per-step reward $r_t = 1 - \lambda\|\delta_t\|^2$: each step alive earns one unit, minus a penalty on the correction. We keep this objective but change the reward so that it is bounded and never negative,

$$r_t = \frac{1}{1 + w\,\|\delta_t\|_2^2/n}, \tag{8}$$

where $n = 16$ is the action dimension, so that the scale of w does not depend on the robot. With an unbounded negative reward, a policy can prefer to fail as fast as possible to stop accruing negative reward (the original ResSafe counters this with a large termination penalty); with $r_t \in (0, 1]$, every step alive is worth something, and an uncorrected step is worth the most. When the object leaves the safe set ($\ell(x) < 0$, i.e., more than 0.3 m from the hand), the episode terminates with no further reward and the environment is reset.

Unlike the original ResSafe, which adds environment-specific terms (rewarding a steady humanoid base) and penalizes the action rate and termination, we add no environment-specific reward terms. The reward is exactly (8), which keeps the formulation simple and general.

2) Run-Time Conservativeness and Training: Unlike the original ResSafe, which trains one filter per correction weight, we make w an input. We draw $w \sim \mathcal{U}(0, 2)$ per environment and episode and give it to both actor and critic as an extra observation, so one network covers the whole range and w becomes a dial at deployment: a large w keeps the executed action close to the nominal policy, and $w = 0$ lets the filter intervene freely.

We use an asymmetric actor–critic whose actor uses only observations available on the real robot: the proprioception and the object pose (o^{prop}, x), plus a^{nom} and w. The critic receives the privileged state in place of (o^{prop}, x). Neither receives the goal. We train the filter without curricula, in the task environment at its final curriculum settings, since the filter must work with the already trained task policy. Also unlike the original, we perturb the nominal action during training with Gaussian noise of a random scale per episode, which diversifies the nominal actions the filter sees, including actions worse than the nominal policy's own. B1 trains with PPO for 4000 iterations on top of the state policy (2.5 h on one RTX 5090) and gives the state filter π_δ. B2 distils it with online DAgger into a depth student π_δ^{d} on top of the depth policy (1.8 h), the depth filter. At deployment we use $w = 0.5$, chosen from the simulated sweep over w (Sec. VI-B.1), unless stated otherwise. Appendix I gives the full description.

3) Baseline: Hamilton–Jacobi Safety Filters: We also compare with another family of learned safety filters, which combines HJ reachability with RL through a safety Bellman backup. It learns a safety critic $Q(x, u)$, which predicts how safe the system stays after taking action u in state x, and a fallback policy π_{safe}, which outputs the action it predicts is safest, with the discounted safety Bellman backup of [**?**]. They are used at run time in one of two

ways. The least-restrictive safety filter (LRSF) executes the task action until the critic predicts a violation and then switches to the fallback action. The human-centered safety filter (HCSF) [**?**], [**?**] instead executes the action closest to the task action that satisfies a discrete-time CBF condition, $Q(x,u) \geq \beta\, Q(x, \pi_{\text{safe}}(x))$, where a larger β makes the filter intervene earlier and more strongly. The action is found by a sampling-based line search over actions between the task and fallback actions. These filters were introduced with SAC [**?**], which is not designed for thousands of parallel simulated environments, so we train them with FlashSAC [**?**]. Its critic is distributional: it predicts a distribution over the safety value. Instead of the mean of this distribution, we use its conditional value at risk CVaR_α, a pessimistic estimate, so α (and β for HCSF) tunes how conservative the filter is, i.e., how much it intervenes. Appendix I gives the background, the equations and our implementation.

VI. Experiments

In our experiments we want to evaluate performance, reliability and safety extensively, stress-testing the system over many trials and external disturbances. We evaluate in simulation and on the real robot: (i) how well the task policies reach goals, how often they drop the object, and how performance and safety interact, e.g., whether the filter lets the robot reach more goals before a drop (Secs. VI-B.1 and VI-C.1); (ii) how ResSafe compares with HJ safety filters (Sec. VI-B.2); (iii) which design choices matter (Sec. VI-B.3); and (iv) further real-robot experiments with spinning, difficult goals, disturbances and downstream tasks that use the policy as a skill (Secs. VI-C.2–VI-D).

A. Evaluation Protocol and Metrics

The simulation benchmark runs 1,024 environments, each with one random wrist orientation held for 50 consecutive goals drawn from the training goal distribution, with a 10 s window per goal. We command no twist for the lightbulb here and test spinning separately (Sec. VI-C.2). In training, the twist and hold conditions mainly encourage stable grasps (the object slow and kept at the goal). For evaluation we only ask whether the object reaches the goal: a goal counts as reached if both e_p and e_R are within the tolerance at any step of its window. We use two tolerances for every object: *coarse* (2 cm, 0.4 rad) and *precise* (1 cm, 0.2 rad), with the tilt angle as the orientation error for the lightbulb and wineglass. A drop is the object leaving the 0.3 m safe set; the environment is then reset to a fixed grasp and continues with the next goal. We report the percentage of goals reached and dropped, the RMS correction of the filter, and the time to the first drop, computed as 10 s times the number of goals completed before an environment's first drop (500 s if it never drops). Table XIV summarizes the protocol of every real-robot experiment.

B. Simulation Results

1) Performance, Safety and the Correction Weight: We evaluate both performance and safety in simulation. For each object, we run the same task policy with and without the safety filter, with the filter at $w \in \{0, 0.5, 1, 1.5, 2\}$ (Fig. 5; Table XVI in Appendix K). With the state policy and $w = 0.5$, the filter lowers the drop rate on every object (knife 3.7 to 2.5 %, lightbulb 6.5 to 2.4 %, rubber duck 9.1 to 4.5 %, wineglass 6.1 to 2.5 % of goals) and delays the first drop (e.g., 154 to 266 s for the duck), while coarse success is maintained or even improves (lightbulb +3.4 and wineglass +2.7 points, knife −0.4, duck −0.8). Precise success is more sensitive: it falls from 88.4 to 80.1 % for the duck and by 3–4 points for the knife and wineglass, while the lightbulb improves. With the depth policy and its distilled filter, drops fall from 4.4–10.2 % to 2.8–3.5 % and the time to the first drop rises from 148–238 s to 273–290 s. Lowering w moves each object along a trade-off curve: $w = 0$ intervenes hardest and gives the fewest drops, at the largest loss of success (for the duck, 70.4 % coarse success with 2.5 % drops), while $w = 2$ stays close to the nominal policy. The lightbulb and wineglass curves are nearly flat, i.e., the filter removes most drops almost for free, whereas the duck pays the most for safety. We deploy $w = 0.5$ on the real robot as a middle ground: it removes most of the drops that $w = 0$ removes, at a much smaller cost in success.

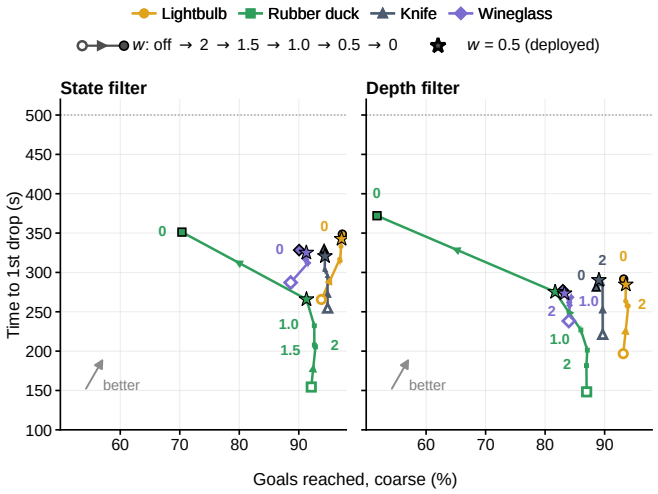


Fig. 5: **Effect of the correction weight w in simulation.** Coarse goal success against time to the first drop, per object, for the state stack (left) and the depth stack (right), from no filter (hollow marker) through $w = 2, 1.5, 1, 0.5$ to 0. Lowering w makes the filter more conservative: drops come later, at the cost of goal success.

2) Comparison with HJ Safety Filters: Table I compares ResSafe with the HJ filters (Sec. V-B.3) on the rubber duck with the state policy; both read the same deployable, goal-free observation at run time. Only ResSafe's critic receives privileged state, since only its actor runs at deployment, whereas the HJ filter uses its critic at run time. ResSafe at $w = 0.5$ halves drops (from 9.1 to 4.5 %) while reaching 91.3 % of goals, within one point of the unfiltered policy. From a sweep over the HJ filter's α and β, the table shows the settings closest to ResSafe's drop rate, and each is worse.

TABLE I: Comparison of ResSafe with the HJ filters (HCSF, LRSF) in simulation, rubber duck, state policy (1,024 environments × 50 goals). Settings: the HCSF settings nearest ResSafe's drop rate from the full sweep (Table XIII) and the best LRSF. Coarse and precise: goals reached (%); drops: % of goals; time to the first drop, or 500 s if none; last column: RMS filter correction (% of the action range). Mean ± s.e. across environments.

Obs.	Filter	Goals reached (%) ↑ coarse	Goals reached (%) ↑ precise	Drops (%) ↓	Time to 1st drop or end (s) ↑	Avg. action change (%)
State	None	92.1 ±0.3	88.4 ±0.3	9.1 ±0.3	154 ±5	–
	ResSafe, w=0.5	91.3 ±0.3	80.1 ±0.4	4.5 ±0.3	266 ±6	15.3
	HCSF α=0.3, β=0.9	88.2 ±0.4	79.8 ±0.5	5.9 ±0.2	199 ±6	10.3
	HCSF α=0.2, β=0.9	83.1 ±0.5	72.1 ±0.6	4.6 ±0.1	227 ±6	15.8
	HCSF α=0.3, β=0.95	81.6 ±0.4	69.3 ±0.5	3.9 ±0.1	253 ±6	17.3
	LRSF α=0.1	74.6 ±0.7	65.5 ±0.8	6.5 ±0.2	170 ±5	19.3

At ResSafe's drop rate (HCSF, $\alpha = 0.2$, $\beta = 0.9$: 4.6 %), HCSF reaches 8 points fewer goals at both tolerances; the setting closest in success drops more often and sooner; and the hard-switching LRSF reaches only 74.6 % of goals with more drops. The HJ filter can push drops lower only by blocking most goals, needs a run-time search over actions, and is sensitive to α and β, whereas ResSafe is a single forward pass whose trade-off is set by one input. For each filter we report the best of three training seeds: for ResSafe the deployed seed, and for the HJ filter the seed with the fewest drops at matched success over all swept settings. Appendix I discusses every setting and the training budgets.

3) Ablations: We ablate the main design choices in simulation: the curricula, the privileged actor observations of A1, and two design choices of the filter.

Curricula. On A1, the friction curriculum contributes the most (Fig. 6; all four objects in Appendix K, Fig. 28). On the knife, over 3 seeds, goal + friction reaches 2.27 goals per episode, friction only 2.14, no curriculum 1.87 and goal only 1.33, and on the lightbulb and rubber duck the friction curriculum gives 15–64 % more goals than none. On the wineglass, no curriculum ends highest at this budget, so the benefit is not universal. A gravity curriculum [6], [2] does not help in our setting (1.11–1.14 goals per episode on the knife), likely because the grasp resets and diverse wrist orientations already make the early task learnable.

Privileged actor observations. Training the A1 actor on privileged observations is essential (Sec. V-A.4): on the knife, an actor that sees only the student's noisy, delayed observations plateaus at 1.12 goals per episode against 2.27 for the privileged actor, over 3 seeds each (Fig. 30).

ResSafe design. On the rubber duck, over 3 seeds, an L1 correction penalty instead of the squared L2 penalty makes the filter all-or-nothing: at $w = 0.5$ it reaches only 21.7 % of goals (coarse) against 80.4 % for L2, and at lower w it takes over and stops the task, so the quadratic penalty is what makes w a usable dial. Removing the noise on the nominal action during training does not change the simulation metrics beyond the seed spread; we keep it for the transfer to the depth policy and the real hand (Appendix K).

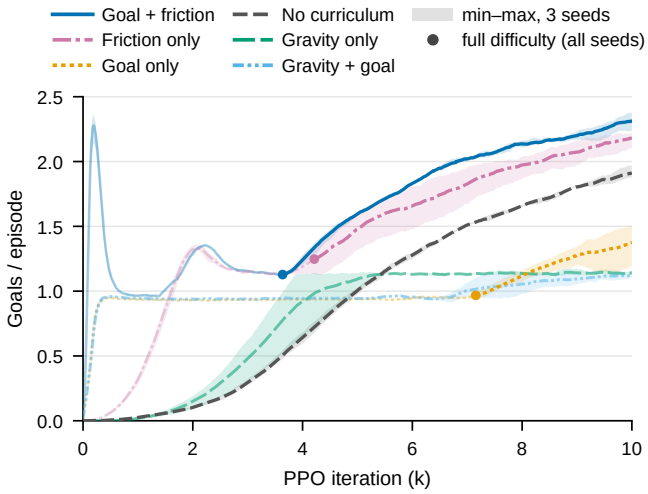


Fig. 6: **Knife curriculum ablation over 3 seeds.** Goals per episode during A1 for each curriculum variant, including a gravity curriculum (gravity 0→9.81 m/s^2, alone or with the goal curriculum). Lines are the mean with a min–max band over 3 seeds, and dots mark full difficulty for all seeds. Final goals per episode: 2.27 (goal + friction), 2.14 (friction only), 1.87 (none), 1.33 (goal only), 1.14 (gravity only) and 1.11 (gravity + goal). Most gravity runs do not reach full gravity within 10k iterations, so they are drawn faded throughout.

C. Real-Robot Results

1) Performance and Safety over 4,800 Goals: We first test performance, reliability and safety over long sequences of goals at six wrist orientations that point gravity along each axis of the hand (Fig. 7). We run the state policy with and without the state filter and the depth policy with and without the depth filter, both at $w = 0.5$, the middle ground from the simulated sweep, with motion capture in every trial, as the state policy's input and as ground truth. For each object and orientation, a sequence of 50 goals drawn with a fixed seed, identical for every variant, comes from a held-out evaluation dataset of 100,000 unseen grasps, and each goal has a 10 s window. After a drop, when the object gets stuck, or when the hand disconnects, we reset the hand to a fixed grasp and continue with the run (Appendix J). We keep the lightbulb's twist command at zero and study spinning separately (Sec. VI-C.2). The 4 variants and 4 objects attempt 4,800 goals, 13.3 h of goal windows on the real robot (Table II, Fig. 22).

Table II and Fig. 22 give the results. Without the filter, the state policy reaches 95.2 % of all goals at the coarse tolerance and 75.6 % at the precise one. Ordered by performance, the objects are lightbulb > rubber duck > knife > wineglass. The wineglass, trained in simulation on looser success tolerances ($\tau_p = 1.5$ cm, $\tau_R = 0.3$ rad), reaches only 141 of 300 goals at the precise one. For each object, coarse success is similar to simulation, and drops are, if anything,

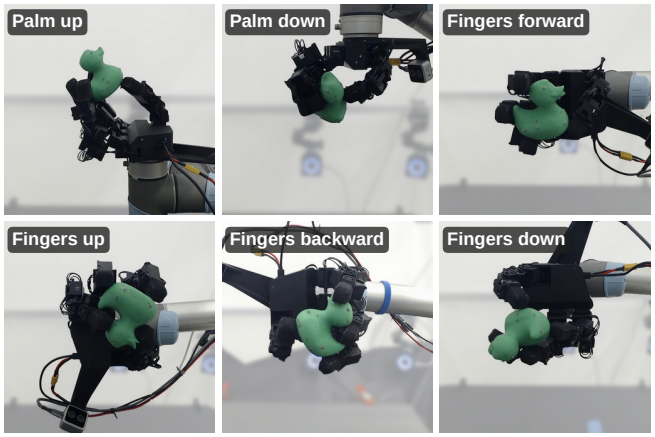


Fig. 7: **The six evaluation wrist orientations**, which point gravity along each axis of the hand.

rarer, although the protocols differ slightly (Appendix J). The depth policy matches simulation on the lightbulb and rubber duck (95 and 88 % coarse success) but not on the knife and wineglass (54 and 50 %, against 90 and 84 % in simulation), and the wineglass drops often (51 of 300 goals). To see whether this gap is perceptual, we compare the depth policy's auxiliary pose estimate with motion capture, replaying the recorded stereo pairs: for the duck and lightbulb it is as accurate as in simulation, but for the knife and wineglass the real errors have long tails that simulation does not show (Appendix K.1), which may explain their larger drop in performance.

Goal by goal, the filter costs some performance: with the state policy, coarse success falls from 95.2 to 91.5 % and precise success from 75.6 to 67.2 %; with the depth policy, from 71.8 to 66.7 % and from 38.3 to 31.8 %. This is expected, since the filter does not see the goal and is never rewarded for reaching it. In safety, and in how safety ties into performance, it gives large gains. With the state policy, drops fall from 16 to 4 of 1,200 goals, the mean time to the first drop rises from 334 to 465 s, and the goals reached before the first drop from 31.8 to 42.9 of 50. With the depth policy, drops fall from 88 to 46, the time to the first drop rises from 228 to 354 s, and the goals reached before the first drop from 17.9 to 26.0. Excluding the depth wineglass, the depth policy's drops fall from 37 to 6. With the filter, the robot thus reaches more goals before a drop and runs longer without one, although it reaches slightly fewer goals overall (Fig. 8, Appendix K). The depth wineglass is the exception: its drops only fall from 51 to 40, again a sign of its visual sim-to-real gap.

Qualitatively, we observe that many drops, in simulation and on the real robot, are hard to anticipate. They do not happen only in visibly difficult grasps or on hard goals: an object in a seemingly stable grasp can suddenly slip after a barely visible change in the finger motion. Whether an action leads to a drop can thus depend on subtle details of the contact, which makes the safety problem hard. This may be one reason why the residual filter, which corrects every action

TABLE II: Real-robot results. Each row pools 6 runs of 50 consecutive goals (one per wrist orientation), 300 goals. Precise and coarse: goals reached within 1 cm, 0.2 rad and 2 cm, 0.4 rad. Drops: goals in which the object fell. Time to the first drop (or 500 s, the end of the run) and coarse goals reached before the first drop: means over the 6 runs. Last column: RMS filter correction (% of the action range). Bold: better of each pair. The filter cuts drops at a small cost in success.

Obs.	Object	Safety filter	Goals reached (/300) ↑ precise	Goals reached (/300) ↑ coarse	Drops (/300) ↓	Time to 1st drop or end (s) ↑	Reached before 1st drop (/50) ↑	Avg. action change (%)
State	Lightbulb	×	**277**	**297**	2	427	42.3	–
		✓	268	292	**0**	**500**	**48.7**	8.5
	Rubber duck	×	**255**	**291**	7	298	29.3	–
		✓	194	262	**2**	**441**	**39.2**	15.4
	Knife	×	**234**	**288**	4	257	24.7	–
		✓	216	281	**1**	**470**	**44.5**	9.3
	Wineglass	×	**141**	**266**	3	352	30.8	–
		✓	128	263	**1**	**449**	**39.2**	8.9
Depth	Lightbulb	×	**191**	**284**	3	371	35.2	–
		✓	183	278	**0**	**500**	**46.3**	7.4
	Rubber duck	×	**152**	**264**	12	268	23.5	–
		✓	95	233	**2**	**392**	**30.3**	13.1
	Knife	×	**60**	**163**	22	163	9.0	–
		✓	53	146	**4**	**406**	**22.0**	6.6
	Wineglass	×	**56**	**151**	51	111	3.7	–
		✓	50	143	**40**	**117**	**5.5**	6.6

a little, compares well with the HJ filters, which must predict when an action is unsafe before intervening. In practice, the knife and wineglass depth policies are still quite usable: this benchmark of random goals is difficult, and with easier goals they still produce remarkably dexterous behavior.

2) Spinning the Lightbulb at a Goal: We also test spinning the lightbulb about its own axis with the depth policy (Fig. 9). The bulb is symmetric, so its full pose cannot be seen and 6D goal reaching is impossible, but with our axis goal and a target twist the policy can still spin it about that axis. In runs of 50 goals of 20 s, each with a random spin command of ± 1 rad/s, the policy reaches 46 and 44 goals with the palm up and down, makes at least one full turn in 28 and 12 of them, and averages 1.24 and 0.63 turns per goal (Table XVIII). The filter costs some performance (43 and 41 goals reached, 0.74 and 0.36 turns per goal), and no run dropped the bulb. Spinning with the palm down is much harder, and at some bulb poses the hand can barely rotate it, which may be a hardware limitation as much as a policy one. Even so, with or without the filter, the policy does not become less safe: when spinning is too hard, it reaches the goal axis and simply holds the bulb still.

3) Difficult Out-of-Distribution Goals: To stress-test the system, we generate a second evaluation dataset of grasps in a 1.5× larger box in front of the palm; only 35–44 % of its grasps lie inside the original box, so the goals lie

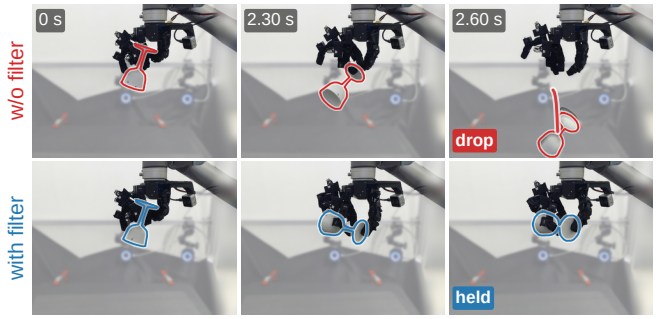


Fig. 8: **ResSafe prevents a drop.** The same wineglass goal (palm down, depth policy) without the filter (top), where the glass slips and falls, and with it (bottom). The red line traces the falling glass.

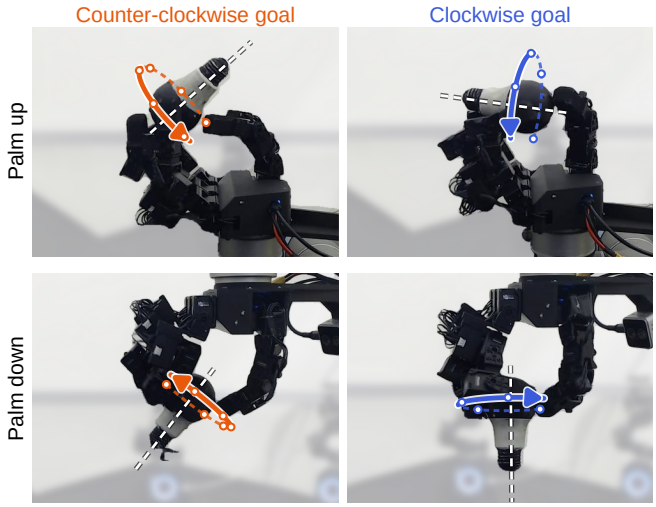


Fig. 9: **In-hand twisting of the lightbulb about its own axis** (depth policy). Palm up (top) and down (bottom), for counter-clockwise (left) and clockwise (right) twist goals, viewed from the screw base toward the glass. Dashed line: bulb axis; arrow: the measured rotation (dots 1 s apart).

largely outside the training distribution (Fig. 23). With the palm down and the state policy, coarse success falls to 28–42 of 50 goals per object (56–84 %), and to 19–39 with the filter (Table XV). Safety, however, does not suffer: only the wineglass dropped, twice with and twice without the filter. Fig. 10 shows some of the very hard goals that were reached, such as holding the knife by the very tip of its handle or the lightbulb without the thumb.

This points to a property of the nominal policy itself: it already acts fairly safely. Dropping the object is never advantageous under our reward, and we observe that the policy does not push itself on near-impossible goals: even with deliberately harder goals, it does not drop more. Yet it still drops the object occasionally, and these drops are not necessarily on the hardest goals but often unexpected. The filter does not intervene sharply at such moments. Instead it applies a small correction at every step that makes the motion slightly more conservative, which prevents some of these unexpected drops.

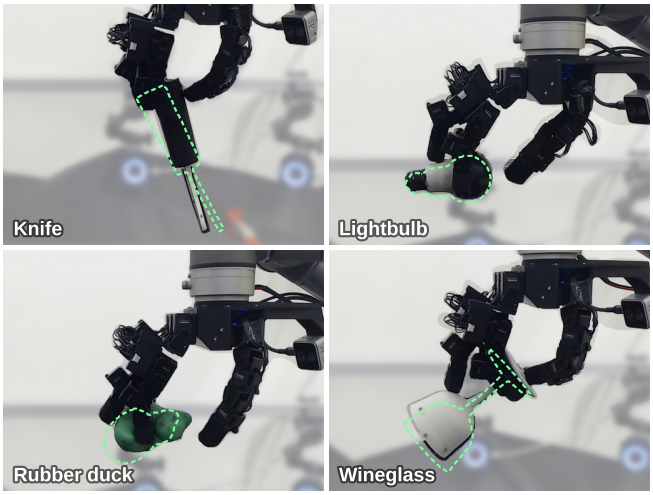


Fig. 10: **Difficult goals reached on the real robot** (palm down, state policy; dashed green outline: goal).

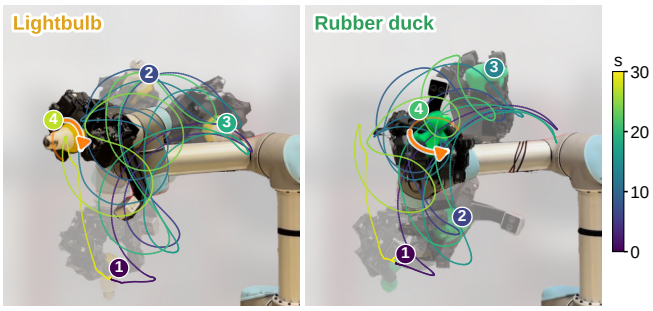


Fig. 11: **Holding in-hand goals during large arm motion.** During a 30 s wrist sweep, the depth policy keeps the lightbulb twisting (with the filter) and the duck spinning (without it). Four poses are shown, with the object path colored by time; the orange arrow is the commanded rotation.

4) Robustness to Unseen Arm Motion: Next, we move the arm while the depth policy spins an object in hand (Fig. 11). This sweeps gravity through most directions in the hand frame and adds inertial forces, neither of which occurs in training, which tests whether the hand keeps working without drops under large disturbances. After 5 s of settling, the three UR5 wrist joints follow sinusoids for 30 s (up to $\pm 85°$, peak 2.75 rad/s), while the lightbulb spins at its goal and the rubber duck's goal rotates continuously. Over 10 trials per setting at two goal heights, the object stays in hand in 32 of 40 trials without the filter and 38 of 40 with it, with the largest gains at the higher, harder goals (Table XIX). At the high lightbulb goal, the filter sometimes keeps the bulb but stops it rotating (3 of 10 trials), apparently overcorrecting. More importantly, the policy keeps spinning the objects under large disturbances it never saw in training, and the filter makes it safer still.

5) Qualitative Robustness: We also find that the depth policy is robust to, and recovers from, physical disturbances it never experienced in training: when a person pulls fingers

of the hand away from the object it holds, the remaining fingers keep the object near the goal, and when a person pulls the object into the palm, the policy regrasps it and returns it to the goal within a few seconds (Appendix K, Fig. 12).

D. Downstream Tasks without Motion Capture

We then use the depth policy as a skill in multi-step tasks, with the hand on the arm and hard-coded arm motions, using only the wrist camera: no motion capture, no external cameras and no markers on the objects (Fig. 13, Table XX). The hand picks up a wineglass and flips it 180° in hand and back; regrasps a lightbulb lying in the palm, carries it under a socket and screws it in with a continuous twist goal until it lights up; and grasps a rubber duck lying on its side, reorients it and places it upright. The in-hand goals of each stage are hand-picked. These tasks contain much that the policy never saw in training: it closes into a grasp from a hard-coded pregrasp next to an object on the table, and once the bulb catches the socket thread, part of it is fixed to the world. The wineglass task succeeds in 7 of 10 trials without the filter and 6 of 10 with it, the lightbulb lights up in 4 of 5 and 3 of 5, and the duck ends upright in 7 of 10 and 3 of 10. The filter, at $w = 1.5$ here, does worse; the reasons are hard to tell, but picking up objects and screwing in the bulb lie far outside what either the policy or the filter saw in training. Object placement and the scripted arm and pregrasp motions add their own failures. The tasks mainly show what a pose-reaching skill makes possible: it needs only a wrist camera and works with clutter in the background of the depth image (Fig. 14). Future work could combine such goal-reaching skills with higher-level planners, such as VLM agents or vision-language-action models.

VII. CONCLUSIONS

We present a recipe that trains a LEAP hand in simulation, without human data, to move objects to arbitrary poses in the hand at any wrist orientation, and that transfers zero-shot to the real robot with motion capture or a single wrist depth camera. On top of it, a learned residual safety filter, trained separately and blind to the goal, greatly reduces drops, and its single dial w trades performance for safety without retraining. Over 13.3 hours of real-robot evaluation, these policies reach almost all goals and drop the object rarely, and because the depth policy needs only a wrist camera, it serves as a portable skill in multi-step tasks. Together, pose reaching and safety filtering offer a practical building block for dexterous manipulation and a platform for studying performance and safety in high-dimensional systems.

Limitations and future work. Our policies are trained per object, and our filter is trained per object and per nominal policy. The depth policies still show a visual sim-to-real gap on the knife and wineglass. Drops remain hard to anticipate and depend on how the policy moves, not only on which goals it attempts, so training the task policy together with the filter, so that it learns safer motions without sacrificing performance, is a natural next step. Future work also includes reaching near 100 % success with zero drops, multi-object policies, closing the visual gap, and extending learned residual safety filters into a general recipe for high-dimensional systems that can filter any nominal policy. We hope this system serves as a first step and a platform to further investigate these questions.

REFERENCES

[1] H. Qi, B. Yi, S. Suresh, M. Lambeta, Y. Ma, R. Calandra, and J. Malik, "General in-hand object rotation with vision and touch," in *Conference on Robot Learning*, 2023. [Online]. Available: https://haozhi.io/rotateit/

[2] T. Chen, M. Tippur, S. Wu, V. Kumar, E. Adelson, and P. Agrawal, "Visual dexterity: In-hand reorientation of novel and complex object shapes," *Science Robotics*, vol. 8, no. 84, p. eadc9244, 2023. [Online]. Available: https://taochenshh.github.io/projects/visual-dexterity

[3] K. Kedia, T. G. W. Lum, J. Bohg, and C. K. Liu, "SimToolReal: An object-centric policy for zero-shot dexterous tool manipulation," *arXiv preprint arXiv:2602.16863*, 2026. [Online]. Available: https://arxiv.org/abs/2602.16863

[4] J. Lin, T. Wu, J. Yin, J. Pan, K. Zhang, and W. Zhi, "Learning in-hand object reaching to general 6D poses," *arXiv preprint arXiv:2609.13761*, 2026. [Online]. Available: https://junxiaolin.github.io/poise-website/assets/papers/poise.pdf?v=08cdf2b28895

[5] P. Yin, T. Westenbroek, Z. Zhang, J. Tran, I. Dagnino, E. Shilamkar, N. Mbiziwo-Tiapo, S. Bagaria, X. Liu, G. Mullins, A. Kolobov, and A. Gupta, "Emergent dexterity via diverse resets and large-scale reinforcement learning," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://weirdlabuw.github.io/omnireset/

[6] P. Li, Z. Chen, Y. Wu, P. Wei, Y. Li, T. Wang, J. Shi, M. Yu, B. Jia, S. chun Zhu, T. Liu, and S. Huang, "Towards human-level dexterous teleoperation," *arXiv preprint arXiv:2607.11481*, 2026. [Online]. Available: https://arxiv.org/abs/2607.11481

[7] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," *arXiv preprint arXiv:1903.11199*, 2019. [Online]. Available: https://arxiv.org/abs/1903.11199

[8] W. S. Cortez, D. Oetomo, C. Manzie, and P. Choong, "Control barrier functions for mechanical systems: Theory and application to robotic grasping," *arXiv preprint arXiv:1903.09816*, 2019. [Online]. Available: https://arxiv.org/abs/1903.09816

[9] A. Wu, R. Bonalli, T. Lew, and C. K. Liu, "Safe and steerable geometric motion policies for robotic dexterous manipulation," *arXiv preprint arXiv:2605.21811*, 2026. [Online]. Available: https://tml.stanford.edu/safe-pbds/

[10] B. Thananjeyan, A. Balakrishna, S. Nair, M. Luo, K. Srinivasan, M. Hwang, J. E. Gonzalez, J. Ibarz, C. Finn, and K. Goldberg, "Recovery RL: Safe reinforcement learning with learned recovery zones," *arXiv preprint arXiv:2010.15920*, 2020. [Online]. Available: https://arxiv.org/abs/2010.15920

[11] J. Seo, K. Nakamura, and A. Bajcsy, "Uncertainty-aware latent safety filters for avoiding out-of-distribution failures," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305, 2025, pp. 4442–4472. [Online]. Available: https://proceedings.mlr.press/v305/seo25a.html

[12] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, "Residual reinforcement learning for robot control," *arXiv preprint arXiv:1812.03201*, 2018. [Online]. Available: https://arxiv.org/abs/1812.03201

[13] K. Shaw, A. Agarwal, and D. Pathak, "LEAP Hand: Low-cost, efficient, and anthropomorphic hand for robot learning," in *Robotics: Science and Systems XIX*, 2023. [Online]. Available: https://roboticsproceedings.org/rss19/p089.html

[14] B. Wen, S. Dewan, and S. Birchfield, "Fast-FoundationStereo: Real-time zero-shot stereo matching," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. [Online]. Available: https://nvlabs.github.io/Fast-FoundationStereo/

[15] M. Duclusaud and G. Passault, "BAM: Better actuator models," Software and documentation, 2024, accessed: October 8, 2026. [Online]. Available: https://bam.readthedocs.io/en/latest/index.html

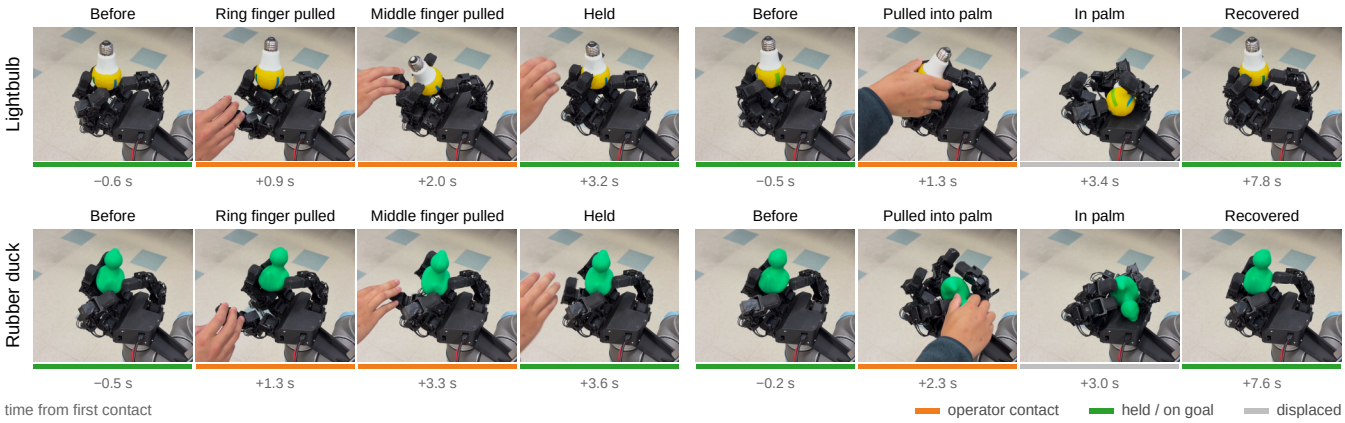


Fig. 12: **Robustness to physical disturbances (depth policy, no filter).** Left: a person pulls two fingers away and the remaining fingers keep the object on goal. Right: a person pulls the object into the palm, and after release the policy returns it to the goal within about 4 s. Bars: human contact (orange), on goal (green), displaced (grey).

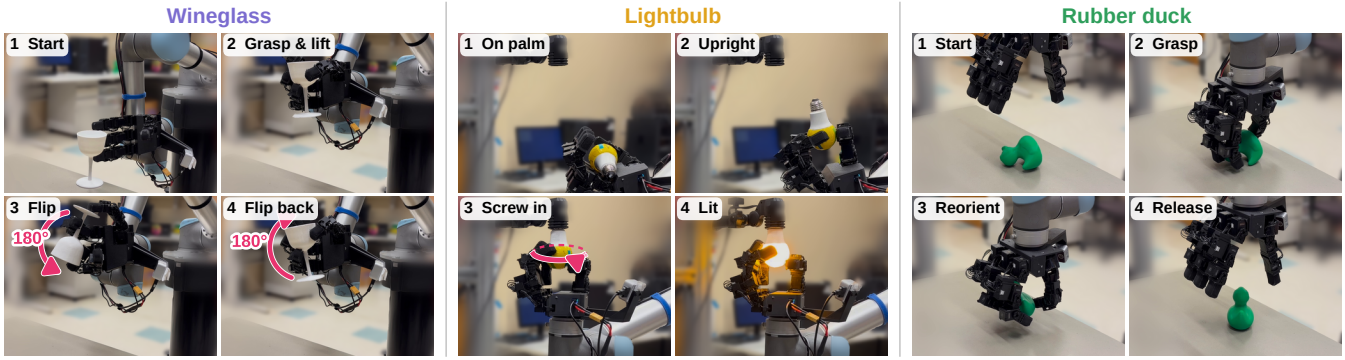


Fig. 13: **Long-horizon arm + hand tasks with only the wrist camera** (depth policy and safety filter). Wineglass: lifted, flipped 180° in hand and back. Lightbulb: turned upright, screwed into the socket under a twist goal, and lit. Rubber duck: grasped from its side, reoriented and placed upright.

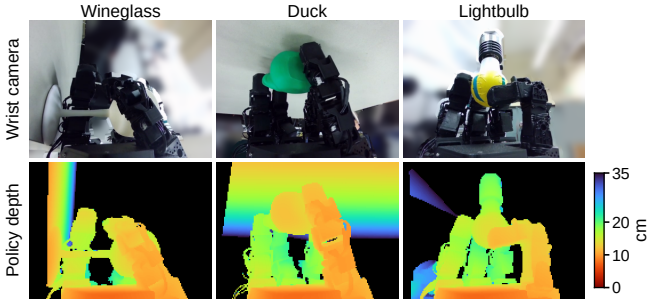


Fig. 14: **Real wrist-camera input next to clutter.** Wrist camera image (top) and the range-gated depth the policy receives (bottom).

[16] K. Zakka, Q. Liao, B. Yi, L. L. Lay, K. Sreenath, and P. Abbeel, "mjlab: A lightweight framework for GPU-accelerated robot learning," *arXiv preprint arXiv:2601.22074*, 2026. [Online]. Available: https://arxiv.org/abs/2601.22074

[17] J. F. Fisac, N. F. Lugovoy, V. Rubies-Royo, S. Ghosh, and C. J. Tomlin, "Bridging Hamilton-Jacobi safety analysis and reinforcement learning," in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 8550–8556. [Online]. Available: https://ieeexplore.ieee.org/document/8794107

[18] M. Duclusaud, G. Passault, V. Padois, and O. Ly, "Extended friction models for the physics simulation of servo actuators," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 12 091–12 097. [Online]. Available: https://bam.readthedocs.io/en/latest/index.html

[19] H. Charlesworth and G. Montana, "Solving complex dexterous manipulation tasks with trajectory optimisation and reinforcement learning," in *International Conference on Machine Learning*, 2021. [Online]. Available: https://dexterous-manipulation.github.io/

[20] X. Liu, H. Wang, and L. Yi, "DexNDM: Closing the reality gap for dexterous in-hand rotation via joint-wise neural dynamics model," *arXiv preprint arXiv:2510.08556*, 2025. [Online]. Available: https://arxiv.org/abs/2510.08556

[21] PKU-Alignment, "ReDMan: Reliable dexterous manipulation," GitHub repository, accessed: October 8, 2026. [Online]. Available: https://github.com/PKU-Alignment/ReDMan

APPENDIX

The appendix gives the full description of the system and additional results: the hardware and depth pipeline (A), system identification (B), the simulation (C), domain randomization (D), the grasp datasets (E), the task environment (F), curricula (G), training (H), the safety filters (I), the real-robot evaluation (J) and additional results (K). Where the main text gives a condensed version, the full original description is here.

A. Hardware and Depth Pipeline

Tables III and IV summarize the real-robot setup and the depth pipeline (Sec. IV-A).

Our robot setup is shown in Fig. 2. We use a LEAP hand [13], nearly identical to the original but with a redesigned palm that makes the thumb joint more stable. It has 16 joints, 4 per finger, each actuated by a Dynamixel XC330-M288 servo in its firmware's position-control mode, on a 4 Mbaud bus; the driver reads all 16 joints at up to 200 Hz. We cover only the fingertips with 3M grip tape, since the bare 3D-printed tips have too little friction for fingertip grasps. The rest of the hand surface is left as 3D-printed plastic, the plastic servo housings and some (non-grippy) electrical tape over exposed wiring. This keeps the policy from relying on a high-friction palm: we prefer it to manipulate mostly with its fingertips, since the wiring on the rest of the hand is not modeled in simulation. Our fingertips are thus hard and rigid: 3D-printed plastic under a thin layer of grip tape. Visual Dexterity [2] designed soft fingertips, which raised its success on unseen objects from 45 to 55 %; we found that our training recipe reaches very good performance even with hard fingertips. An Orbbec Gemini 305 stereo camera is mounted on the wrist with a 3D-printed adapter, looking at the palm and fingers. It streams both color sensors at 640×480 and 60 fps. Instead of the depth computed by the camera's own software, we run Fast FoundationStereo [14] (TensorRT, FP16) on the rectified pair, which we found gives much denser and less noisy depth, especially for small, close-range objects; the full capture and stereo pipeline runs at 30 Hz, and the depth image is downsampled to 160×120 for the policy (Table IV). The hand is mounted on a UR5 arm, which is not controlled by any learned policy: it only moves the hand to different positions and wrist orientations, and its joint readings (50 Hz over RTDE) give the direction of gravity in the hand frame. A motion-capture system (OptiTrack, about 100 Hz) serves both as the object-pose input for the state policies and as ground truth for evaluation. For this we have a second copy of each real object carrying 10–12 small (3 mm) markers, so that the object frame is tracked even under heavy finger occlusion, and 9 markers on the hand, so the object pose can be expressed relative to the hand root. The policy runs at 20 Hz and sends joint position targets to the servos, whose firmware runs a P-only position loop with a PWM limit. The objects are 3D printed except the lightbulb, which is a real bulb. For the depth policies we cover the bulb's glossy glass with duct tape (not a high-friction surface), because the stereo network estimates the depth of reflective glass poorly, and draw a few stripes on the tape so that its rotation is easy to see. For the motion-capture copy we instead paint the glass and the shiny metal screw base black, since reflections cause false marker detections; the paint keeps the surface about as slippery as plastic, so it does not make the bulb easier to manipulate.

TABLE III: Real-robot setup. Measured latencies are from the system-identification recording; the simulation ranges are the randomized delays (Tables VIII, V).

Hardware	
Hand	LEAP hand, 16 Dynamixel XC330-M288 servos in position mode (Table V); 4 Mbaud bus, joints read at up to 200 Hz
Arm	UR5 (RTDE, 50 Hz); only places and orients the hand (6 taught orientations), never commanded by the policy; its pose gives the gravity direction
Object pose x	OptiTrack, ≈100 Hz; object and hand rigid bodies, converted to the hand frame, unfiltered (state policies only)
Depth I	wrist-mounted Orbbec Gemini 305 stereo + Fast FoundationStereo (Table IV)
Compute	one PC (2× NVIDIA RTX 4080 SUPER); policies (TorchScript) and stereo network on one GPU
Control loop	
Rate	20 Hz, deadline-scheduled; each device on its own thread (latest value)
Inference (median)	5.0–5.7 ms; 5.6–7.4 ms with ResSafe
ResSafe	$a = \mathrm{clip}(a^{\mathrm{nom}} + \delta, -1, 1)$, $w = 0.5$
Stops	joint reading older than 0.25 s, object pose older than 0.5 s, filter error
Measured latency (simulation range)	
Joint lag	17.5 ms command to reading; reading age 4.9 ms (0–50 ms)
Actuation	12.6 ms command to motor (0–33 ms)
Depth	75 ms end to end (0–100 ms)

All computation runs on one desktop: the whole robot stack, including the stereo network, runs on one of its two NVIDIA RTX 4080 SUPER GPUs. Each device (hand bus, arm, motion capture, camera and stereo network) runs in its own thread and keeps only its latest reading, and the 20 Hz control loop is scheduled on absolute deadlines, so a late step never makes later steps run faster. Policy inference, including the safety filter, takes a median of 5.0 ms for the state policy (5.7 ms with the filter) and 5.7 ms for the depth policy (7.4 ms with the filter). The loop stops the hand if a joint reading is older than 0.25 s or the object pose is older than 0.5 s. Table III summarizes the setup.

B. System Identification

Table V lists the identified actuator model and its randomization, Fig. 15 shows the test bench and the fit, and Fig. 16 compares the identified simulation with the real hand (Sec. IV-B).

Actuator model. Simulators usually model a position-controlled joint as an ideal PD actuator, which turns the position command into a torque $\tau = k_p(q^* - q) - k_d\dot{q}$ with hand-tuned gains. Instead, we model the servo itself. We identify the XC330 with the BAM framework [**?**], [15], which models a voltage-controlled DC motor with a detailed friction model, on a single-servo pendulum test bench (Fig. 15): 240 logs of about 6 s each, covering 10 mass/arm-length setups (0.056–0.296 kg, 0.08–0.15 m), 6 trajectory types (sinusoids, chirps, steps, lift-and-drop) and 4 firmware P gains. We fit BAM's m5 model, a voltage-driven DC-motor model with Coulomb,

TABLE IV: Depth pipeline. The policy samples the newest frame at 20 Hz; total delay to the policy is 75 ms. The simulated camera renders depth directly at 160×120 with the same intrinsics and goes through the same range gate, plus the augmentation in Table VIII.

Camera	
Sensor	Orbbec Gemini 305, palm-mounted
Stereo pair	two colour sensors, 640×480 at 60 fps, hardware-synced, fixed exposure
Baseline	18.0 mm
Rectification	factory calibration, rectified into the nominal intrinsics f_x=371.0, f_y=370.8, c=(321.9, 238.5) px
Stereo network	
Model	Fast FoundationStereo (ViT-L), TensorRT FP16
Input	480×640, 8 iterations, max. disparity 192 px
Throughput	26 ms per frame, capped at 30 Hz
Processing	
Depth	$z = f_x B/d$
Range gate	outside $(0.001, 0.35)$ m or invalid → 0 (as in sim)
Resize	nearest neighbour to 160×120 (intrinsics /4, vertical FoV 65.8°)
Normalization	$[0.001, 0.35]$ m → $[0, 1]$, inside the policy (as in sim)
Filtering	none (no hole filling)

viscous, load-dependent and Stribeck friction. Replayed open loop, it tracks the real servo with a mean absolute error of 0.77° (2.4° max). Since all 16 joints use the same servo, the same parameters are used for every joint (Table V). We also model the firmware's P-only loop with its PWM limit, the 1.8 A current limit, and the supply voltage drop across the shared bus.

Latency calibration. We play a recorded swept-sine command (0.8 to 2.5 Hz, 30 s) on all 16 joints of the real hand with the palm up and no object, replay the same commands in a delay-free simulation, and find the time shift that best aligns the two. The command-to-reading lag of the joints is 17.5 ms; subtracting the measured 4.9 ms age of a joint reading leaves an actuation delay of 12.6 ms. Aligning the depth stream the same way gives 75 ms end to end. These measurements set the centers of the randomized delays in simulation (Tables VIII and V). With the identified model and delays, simulated joints follow the real ones with a per-joint RMSE of 0.45°, against 1.14° without delays (Fig. 16).

C. Simulation

Fig. 3 (Sec. IV-C) shows the four objects in simulation and reality, with their collision geometry.

We build our simulation environment in mjlab [16], which runs MuJoCo [**?**] physics on the GPU through MuJoCo Warp [**?**]. We do not simulate the arm: the hand is fixed in place in free space, with no floor, and only the direction of gravity relative to the hand changes from episode to episode. The hand has 16 joints, 4 per finger, each actuated by a simulated Dynamixel XC330. We start from the LEAP hand models of [**?**] and MuJoCo Menagerie [**?**] and heavily modify the link masses, inertias and some collision geometry to match our robot. We simulate the wrist camera with mjlab's camera sensor, which uses MuJoCo Warp's ray-traced rendering, with the intrinsics and extrinsics of our real camera; depth is rendered directly at 160×120. The physics timestep is 1/120 s, and the actuator model runs at this rate, while each environment step is 6 physics steps, so the policy runs at 20 Hz as on the real robot. We set up four tasks, each manipulating a single object (Fig. 3, Table VI). MuJoCo treats every mesh geom as its convex hull, and all four objects are concave, so we decompose each object mesh into convex parts with CoACD [**?**] (Fig. 3): 4, 3, 5 and 32 parts for the knife, lightbulb, rubber duck and wineglass. To reduce the sim-to-real gap, the simulated joints are driven by the voltage-based BAM servo model identified above rather than an ideal PD actuator, with the sensing and actuation delays measured on the real hand.

Table VII lists the simulator settings. We use the implicit-fast integrator, the Newton solver with 10 iterations and 20 line-search steps, and elliptic friction cones with an impedance ratio of 10. Episodes last 20 s. Gravity is -9.81 m/s^2 in the world frame; a random hand orientation per episode is what changes its direction relative to the hand. The hand colliders take contact priority over the object, so the hand's friction coefficients determine every hand–object contact, and we randomize only the hand's friction. The fingertip contacts also model torsional and rolling friction. Contact sensors between the fingertips and the object, be-

TABLE V: Actuator model of the LEAP hand's Dynamixel XC330 servos and its randomization. Model parameters are scaled independently per joint at every episode (armature once per environment). PWM limit: MCP joints / other joints. "Load friction" terms are fractions of the load torque.

Parameter	Value	Unit	Randomization
*BAM m5 model [**?**], identified on a test bench*			
Torque constant k_t	0.571	N m/A	$\times\mathcal{U}(0.9, 1.1)$
Winding resistance R	2.327	Ω	$\times\mathcal{U}(0.9, 1.1)$
Armature	1.83×10^{-3}	kg m^2	$\times\mathcal{U}(0.9, 1.1)$
Base friction	4.85×10^{-3}	N m	$\times\mathcal{U}(0.9, 1.1)$
Viscous friction	3.97×10^{-3}	N m s/rad	$\times\mathcal{U}(0.9, 1.1)$
Load friction, motor side	0.421	–	$\times\mathcal{U}(0.9, 1.1)$
Load friction, external Stribeck	0.0997	–	$\times\mathcal{U}(0.9, 1.1)$
Stribeck velocity	0.255	rad/s	$\times\mathcal{U}(0.9, 1.1)$
Stribeck exponent α	1.824	–	$\times\mathcal{U}(0.9, 1.1)$
Stribeck friction	2.65×10^{-9}	N m	fixed
Load friction, external	8.84×10^{-8}	–	fixed
Load friction, motor Stribeck	4.88×10^{-7}	–	fixed
Firmware, encoder and supply			
Position P-gain	1100	–	fixed
Error gain[a]	2.88×10^{-3}	duty/rad	fixed
PWM limit (of 885)	300 / 200	–	fixed
Current limit	1.8	A	fixed
Encoder bias[b]	0	rad	$\mathcal{U}(-0.03, 0.03)$
Supply voltage v_{in}	5.0	V	$\mathcal{U}(4.5, 5.25)$
Bus resistance[c]	0	Ω	$\mathcal{U}(0, 0.4)$
Command channel (physics steps at 120 Hz)			
Command latency[d]	0	steps	$\mathcal{U}\{0, \dots, 4\}$
Dropped-command rate[e]	0	–	3 % per control step
Dropped-command hold	–	steps	$\mathcal{U}\{1, \dots, 24\}$

[a] Duty cycle per unit of P-gain × position error, $(4096/2\pi)/(256 \cdot 885)$ from the encoder resolution, firmware gain divisor and full-scale PWM register: duty = $1100 \cdot 2.88\times10^{-3}(q^* - q)$, clipped to the PWM limit (e.g. at 71 mrad of error for 200/885). [b] Per joint, drawn every episode in all stages. Deployable observations read the biased angle, privileged ones the true angle. Commands are built relative to the measured angle, so the bias cancels in actuation except where targets are clipped at the joint limits. [c] The bus droops as $V = v_{\text{in}} - R_{\text{bus}} \sum_j I_j$, so a hand squeezing hard loses torque. [d] 0–33 ms; one lag per environment, shared by all joints. [e] Action masking [6]: each joint independently misses a command and holds its last target for the sampled number of physics steps (8–200 ms).

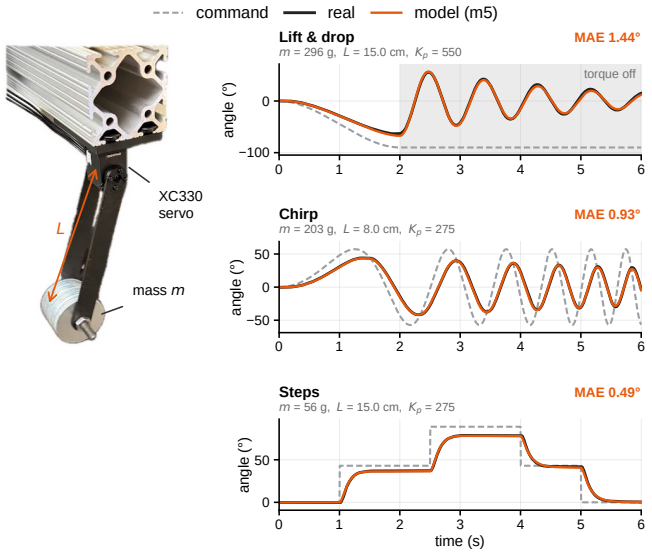


Fig. 15: **Actuator identification of the XC330 servo.** Left: pendulum test bench with mass m at arm length L. Right: commanded angle, real response and the identified BAM m5 model on three test trajectories, each annotated with m, L, firmware gain K_p and the model's MAE. Over all 240 logs (10 mass/length setups $\times$ 6 trajectories $\times$ 4 gains), the model tracks the real servo with 0.77° mean absolute error; the shown logs are the median-error gain of each setup.

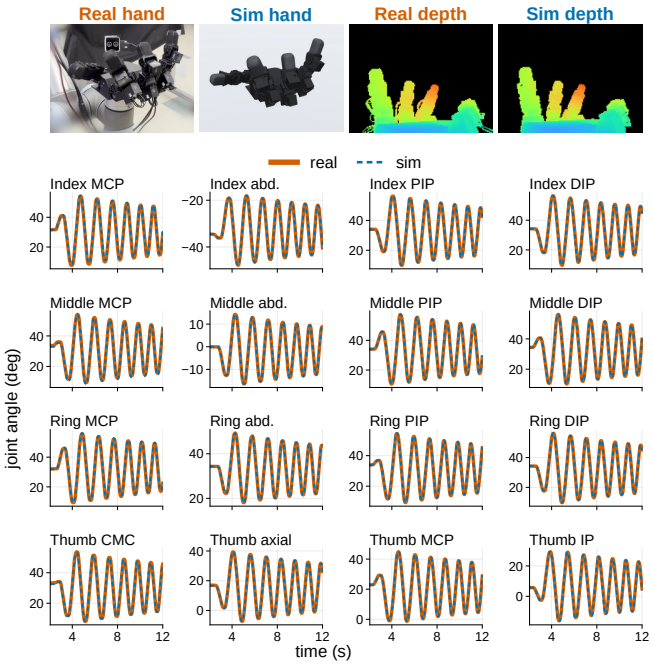


Fig. 16: **Sim-to-real check of the identified hand.** The same recorded swept-sine joint commands are played on the real hand and, open loop, in simulation with the calibrated delays. Top: real and simulated hand and wrist depth at the same instant. Bottom: all 16 joint angles, real (solid) vs. simulated (dashed); per-joint RMSE is 0.45° (1.14° without delays).

tween the other links and the object, and among the hand's own links provide the contact rewards and the fingertip-contact observation. An object's mass is distributed over its convex parts by volume.

D. Domain Randomization

We randomize the simulation broadly (Tables VIII and V). Every episode draws new hand friction coefficients (sliding, torsional and rolling, separately for the fingertips and the other links), actuator parameters (each BAM parameter scaled by $\mathcal{U}(0.9, 1.1)$ per joint), supply voltage, bus resistance and per-joint encoder biases. Each environment draws new link and object masses, inertias and centers of mass, which we perturb jointly through the pseudo-inertia matrix so that they stay physically consistent [**?**], as well as small offsets of the link frames and the robot and object collision geometry. During each episode, random forces push the object [**?**], the commanded targets are delayed and occasionally dropped, and the deployable observations are corrupted with noise and latency matched to the delays measured on the real robot. Privileged observations stay clean.

Table VIII lists the domain randomization shared by every training stage (Sec. IV-C).

E. Grasp Dataset Generation

Grasp dataset. Rather than sampling goals uniformly over $SE(3)$, most of which no grasp can reach, we generate a grasp dataset for each object and our LEAP hand offline, with no human data. We first use LightningGrasp [**?**] to generate 100,000 grasps. It places the object around a canonical box in front of the palm ($x \in [0, 10]$ cm, $y \in [-3.5, 3.5]$ cm, $z \in [4.7, 14.5]$ cm in the palm frame, sampled uniformly), touching the palm for half of the placements. It then searches the object surface for three fingertip contacts whose normals, together with the palm contact if present, balance forces and torques (a frictionless wrench residual below 0.15), and solves inverse kinematics so that the fingertip pads reach these contacts within 4 mm. Contacts are restricted to the distal pads of the fingertips and the inner face of the palm. The remaining finger is set to random joint angles, and grasps with hand–object or hand self-collision are rejected. We then expand these seeds into a dense set of 500,000 grasps per object with the RRT formulation of DexterityGen [**?**]. We repeatedly sample a random hand configuration and object pose (joint angles uniform within the joint limits, object position uniform in the canonical box, orientation uniform over $SO(3)$), find its nearest grasp among 2048 random grasps of the current dataset under a weighted distance over joint angles, object position and object rotation (weights 1, 50 and 3), and step 20 % of the way from that grasp toward the sample (linearly in joint angles and position, by slerp in rotation). The parent's contact points move rigidly with the object, the fingers are re-solved by inverse kinematics, and the new grasp is added to the dataset if it passes the same collision check. Every new grasp therefore keeps its parent's contact points on the object and is a short, kinematically feasible step away from it, so the dataset covers

TABLE VI: Per-object settings. A goal is reached when the object stays within all three tolerances for h_{req} control steps, drawn per goal; goals and initial grasps are drawn from the object's grasp dataset. In A1 the tolerances start at 2 cm, 0.4 rad and 1.5 rad/s and tighten to the values shown. A task episode ends on a drop (object 0.5 m from the hand); ResSafe's safe set is 0.3 m.

	Knife	Lightbulb	Rubber duck	Wineglass
Mass, nominal [g]	40	27	46.7	62
Mass DR, $\log \mathcal{U}$ [g]	20–90	10–50	20–70	30–90
Bounding box [mm]	39×47×210	57×57×103	59×81×85	80×80×137
Goal[a]	pose	axis	pose	axis
Grasps in dataset	451,681	453,131	439,212	421,949
Success tolerance (A1 final, A2, A3, B1)				
Position [cm]	1.0	1.0	1.0	1.5
Orientation [rad]	0.2	0.12	0.2	0.3
Twist [rad/s]	0.35	0.35	0.35	0.35
Success tolerance, depth stages (A4, B2)				
Pos. / orient. [cm / rad]	2.0 / 0.4	2.0 / 0.4	2.0 / 0.4	2.0 / 0.4
Goal command				
Twist rate [rad/s][b]	0	$\mathcal{U}(-1.5, 1.5)$	0	0
Hold h_{req}, $\mathcal{U}\{\cdot\}$ [steps][c]	10–20	10–60	10–20	10–20
Training				
A1 environments (2 GPUs)	16,384	16,384	16,384	32,768

[a] Pose: position + orientation. Axis (z-symmetric objects): position + direction of the object's z-axis, so the orientation error is a tilt angle. [b] About the goal axis; 0 (hold still) with probability 0.5. [c] A leaky counter: +1 per step inside all tolerances, -1 outside; the goal is reached when it reaches h_{req}.

TABLE VII: Simulation settings, shared by all stages and objects.

Setting	Value
Simulator	mjlab 1.5 on MuJoCo Warp 3.10 (GPU)
Physics timestep	1/120 s
Control rate	20 Hz (6 physics steps per action)
Episode length	20 s (400 control steps)
Integrator	implicit-fast
Constraint solver	Newton; 10 iterations, 20 line-search steps
Friction cone	elliptic, impedance ratio 10
Gravity	9.81 m/s^2; its direction in the hand frame is set by the random wrist orientation
Hand	LEAP hand, 16 joints; torque from the actuator model (Table V)
Hand–object friction	set by the hand colliders (contact priority), Table VIII
Object collision	convex parts (Table VI); mass split over the parts by volume

the box densely and its neighboring grasps are reachable from each other. Each grasp stores the 16 joint angles and the object pose in the palm frame. The generator's collision check uses coarse geometry, so we finally filter every dataset against the simulator's own geometry, using batched MuJoCo Warp collision detection in mjlab with the object's convex decomposition. For each grasp we measure the deepest penetration of the hand into the object and the deepest penetration between different fingers or between a finger and the palm, on the simulator's collision model of the hand, on the same model with extra boxes that bridge the gaps across the knuckles (so a thin knife blade cannot slip through a finger unseen), and on the visual meshes. We remove grasps that penetrate the object by more than 8 mm or the hand by more than 5 mm in any of these models. This removes 9.4–15.6 % of the grasps and leaves 422k–453k



Fig. 17: **Initial grasps from the grasp datasets.** Each training episode starts from a dataset grasp (16 joint angles and the object pose in the hand frame, penetration-filtered) under a random hand orientation. Shown are three very different grasps per object.

per object (Table VI, Fig. 17). During training, goals and initial states are drawn from this dataset. Goals are therefore reachable in practice: they are free of deep collisions and pass the generator's force-balance test, although they are not verified by simulating them. After training, the policy also reaches grasps that are not in the dataset, as we show in the experiments.

F. Task Environment

Full description of the task environment (Secs. IV-C and V-A): goal success, resets and terminations, the action space, the observations (Table IX, Fig. 18) and the reward.

Goal success in training. In simulation a goal must satisfy four conditions to count as reached. The position and orientation errors (1) must be below the thresholds τ_p and τ_R. The twist error must be below τ_ω: the lightbulb is commanded a target spin rate $\omega_z^* \sim \mathcal{U}(-1.5, 1.5)$ rad/s about its goal axis (or zero with probability 0.5), and every other object a target of zero. We low-pass filter the object's angular velocity ω, measured in the hand frame,

$$\bar{\omega}_t = (1-\alpha_\omega)\,\bar{\omega}_{t-1} + \alpha_\omega\,\omega_t, \qquad e_\omega = \|\bar{\omega}_t - \omega_z^*\,z^*\|_2, \quad (9)$$

where z^* is the goal axis (the goal's z-axis for pose goals), $\omega_z^* = 0$ for every object except the lightbulb, and $\alpha_\omega = 0.05$ (a 1 s time constant at 20 Hz). The twist condition makes the policy hold the object still (or spin it steadily) instead of passing through the goal. Finally, the goal must be held for h_{req} steps, though not necessarily in a row: a hold counter h, reset to 0 for each new goal, goes up by one at every step that satisfies all three conditions and down by one (but not below 0) at every step that does not, and the goal is reached when h reaches h_{req}. A brief slip out of tolerance thus delays success without restarting the count. h_{req} is drawn per goal from $\mathcal{U}\{10, \ldots, h_{\max}\}$ steps (0.5–1 s, and up to 3 s for the lightbulb). When a goal is reached, the next goal is drawn in the same step: a new grasp from the dataset, whose object position and orientation become the new goal. The objects

differ only in the settings listed in Table VI: mass, goal type (pose or axis), success thresholds, twist range and hold range.

Goal selection. Goals never time out: a new goal is drawn only once the current one is reached, so the number of goals reached per 20 s episode measures both speed and success. Early in training we do not draw the next goal uniformly. Following DexterityGen [**?**], we draw a random subset of N grasps from the dataset, rank them by their distance to the current state,

$$d_i = \|q - q_i\|_2 + 50\,\|p - p_i\|_2 + 2\,\theta(R, R_i), \qquad (10)$$

where q are the measured joint angles, (p, R) is the current object pose, (q_i, p_i, R_i) is grasp i and θ is the geodesic angle, and pick one of the 10 nearest uniformly. Goals are thus nearby and reachable at first. The goal curriculum (Sec. V-A.3) shrinks N from 2048 to 1, so the goals become uniform over the whole dataset by the end of training and in every later stage.

1) Resets and Terminations: Lessons from DexterityGen [**?**], BeyondMimic [**?**], POISE [4], TeleDexter [6] and OmniReset [5] show that targeted, diverse resets guide RL exploration. At every reset, the hand and object therefore start in a grasp drawn uniformly from the whole dataset (joint angles, object pose and its velocity), and the hand's base orientation is drawn uniformly on $SO(3)$, which sets the direction of gravity in the hand frame for the episode. Each episode lasts 20 s (400 control steps). Reaching the time limit is treated as a time-out rather than a failure: PPO bootstraps the value of the last state, so the policy is not penalized for an episode running out [**?**]. An episode terminates when the object is more than 0.5 m from the hand root (a drop), which also cuts off all future reward, and a NaN guard resets an environment if the simulation diverges; a NaN reset is not counted as a drop. The lengths of the first episodes are staggered so that resets are spread out in time across the parallel environments.

2) Action Space: A study of action spaces for RL [**?**] argues that this integrator-style action is not the best choice, and that $q^{\mathrm{cmd}} = q + a$ tracks better, since otherwise the previous target q^{cmd}_{t-1} becomes extra hidden state. We nevertheless found the integrator more reliable for system identification. The commanded target is a deterministic function of the actions alone, so the same command sequence produces the same targets on the real hardware and in simulation. If the target were built from the measured position, it would depend on which reading the real robot happened to use, how stale that reading was, and when it was read. Fig. 16 shows that, with this action, simulation and the real hand track the same swept-sine commands closely. To avoid the target drifting far from the actual position (for example when a finger is blocked by the object, which would otherwise wind up the target and overshoot when released), we clip it to within 0.3 rad of the measured joint position.

Inspired by TeleDexter [6], we also apply action masking at the actuator level: in every control step, each joint independently misses its new command with probability 3 % and holds its last received target for $\mathcal{U}\{1, \ldots, 24\}$ physics steps (8–200 ms), which simulates dropped packets on the shared servo bus.

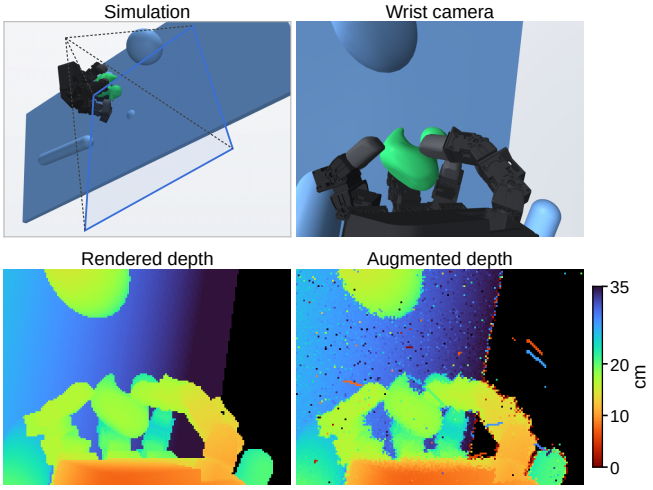


Fig. 18: **Simulated depth observations.** A simulated hand with random distractor objects (scene with the wrist-camera frustum), the rendered wrist camera view, the rendered depth, and the augmented depth that the depth student actually receives.

In addition, a randomized command latency of $\mathcal{U}\{0, \ldots, 4\}$ physics steps (0–33 ms), shared by all joints, is applied before the actuator model (Table V).

3) Observations: We draw from a variety of observations in simulation, listed in Table IX and grouped by the symbols of Fig. 4. The proprioception o^{prop} contains what is available on the real robot: the gravity direction in the hand frame (computed from the arm's end-effector orientation), a 0.5 s history of joint positions read from the servo encoders, the residual between the commanded target and the measured position, and the last action. x is the object pose (from motion capture on the robot) and g is the goal. For the depth policies, I is the 120×160 wrist depth image, with returns outside $(0.001, 0.35)$ m set to zero and the remaining depths normalized to $[0, 1]$. The privileged observations o^{priv} are not available on the robot: noise-free joint, fingertip and object states including velocities, short histories of fingertip and object poses, and fingertip contact flags and forces. The privileged goal g^{priv} adds the goal relative to the object. Finally, some observations are exposed only to the critic, as in asymmetric actor–critic training [**?**]: the randomized fingertip friction and encoder biases, and the success bookkeeping (errors, whether the object is inside the tolerances, hold progress, required hold length, twist error). These help the critic predict returns, but we keep them out of the actor because a policy that could read, e.g., the sampled friction would learn behavior conditioned on information it can never infer on the real robot, and the success bookkeeping would let the actor key on the reward definition rather than on the object state.

4) Reward: We keep the reward simple. Although it has several terms, it consists of dense goal-reaching terms and

a few regularizers. We treat the goal-reaching terms as potentials [**?**]: the policy is rewarded for increasing a potential and penalized by the same amount for decreasing it. This differs from the dense goal-reaching rewards used in other work, such as DexterityGen [**?**], in-hand 6D reorientation [**?**] and TeleDexter [6], which reward closeness to the goal at every step. We did not test that alternative, and it would likely also work, but we believe the potential form avoids a local optimum in which the policy hovers near the goal and collects reward without ever reaching it; this is especially tempting when reaching a goal resamples a new, possibly harder goal. When only progress is rewarded and regress is penalized, staying near the goal earns nothing.

The position potential is capped at 10 cm so that a far-away object does not dominate the reward, and the twist potential only counts once the pose is within tolerance. n^{tip} is the number of fingertips touching the object, n^{link} the number of other links touching it and n^{self} the number of hand self-contacts; the small ± 0.01 terms encourage fingertip grasps and discourage palm and link contact and self-collision. The action-rate and action-magnitude penalties discourage noisy, jerky solutions. The drop penalty is applied once, on the terminating step, and together with the loss of all future goal reward it discourages catastrophic failures. There is no separate success bonus: reaching a goal is rewarded through the hold potential Φ^h, which rises to 1 as the goal is held. When a goal is reached and a new one is drawn, the potentials are reset to the new goal's current values (the hold counter and its potential return to zero, and the position, orientation and twist progress terms are skipped for that one step), so reaching a goal never registers as a loss of potential.

G. Curricula

Fig. 19 shows the schedules of both curricula as a function of their progress, and Fig. 20 their progress during A1 (Sec. V-A.3).

We find that policies can be learned with the design above, but curricula speed up learning dramatically. When training the privileged teacher (A1) we use a goal curriculum and a friction curriculum, both adaptive in the style of automatic domain randomization [**?**], [**?**]: their difficulty depends on how well the policy is currently doing. Inspired by OmniReset [5], we keep a buffer of the outcomes of the last 500 goals across all environments (1 if the goal was reached, 0 if the episode ended first). Every 10 control steps, each curriculum moves its progress $p \in [0, 1]$ up by $1/2048$ if the fraction of reached goals is above its threshold, and down by $1/2048$ if it is below a slightly lower threshold (0.50/0.49 for the goal curriculum and 0.55/0.54 for the friction curriculum). Progress is frozen until the success rate first reaches the upper threshold, so a policy that is still learning the basics is not pushed back to an even easier task. The progress then sets the parameters of each curriculum (Fig. 19, Fig. 20). For the goal curriculum we follow DexterityGen [**?**]: each goal is drawn by the nearest-neighbor rule described above, among a random subset of the dataset whose size shrinks geometrically from 2048 to 1 as p goes from 0 to 1, $N = \text{round}(2048^{1-p})$. Goals start nearby and easy to reach and become more and more random, until with fewer than 10 candidates the goal is drawn uniformly from the whole dataset. The same progress also tightens the success tolerances linearly from 2 cm, 0.4 rad and 1.5 rad/s to each object's final values (Table VI), and raises the maximum required hold $h_{\max}$ from 10 to 20 steps (60 for the lightbulb).

The friction curriculum is simple. mjlab can modify the friction of any collision geom directly on the GPU (in other frameworks such as Isaac Lab this currently goes through the CPU). Friction is domain randomized: at every reset, the friction of each hand geom is drawn uniformly from a range. At $p = 0$ the ranges are unrealistically high: $\mathcal{U}(2.5, 3.0)$ sliding friction for the fingertips and $\mathcal{U}(1.0, 1.5)$ for the rest of the hand. As p increases, the bounds move linearly to the final ranges $\mathcal{U}(0.3, 1.2)$ and $\mathcal{U}(0.0, 0.6)$ at $p = 1$, and the torsional and rolling friction ranges widen geometrically toward zero (Table VIII). The intuition is that people also learn fine manipulation with helpers that make things less slippery and easier to hold. Friction changes the difficulty smoothly, without changing the structure of the environment or the task the policy has to solve.

All later stages (A2–A4, B1–B2) run with both curricula pinned at $p = 1$.

H. Training Details and Compute

Tables X and XI list the hyperparameters of every stage, and Fig. 21 shows the training curves of the four task stages (Sec. V-A.4).

We follow the common recipe of distilling privileged teachers into deployable students, but split it into more stages, in the style of VideoMimic [**?**]: a first RL stage learns the behavior, distillation moves it to deployable observations, and a further PPO stage refines the student. In the first stage (A1), we train with PPO [**?**] using the rsl_rl implementation [**?**], with all privileged information $(o^{\text{priv}}, g^{\text{priv}})$ as actor input and an asymmetric critic $(o^{\text{crit}}, g^{\text{crit}})$. This is also the only stage that uses the curricula. Without the velocities and other privileged inputs in the actor, training was much slower and converged lower: on the knife, an actor given only the student's noisy, delayed observations (with the same critic and curricula) plateaus at 1.12 goals per episode against 2.27 for the privileged actor, over 3 seeds each (Fig. 30). We therefore did not follow the asymmetric actor–critic style of TeleDexter [6] or SimToolReal [3], whose actor is deployable directly on the real robot. In the second stage (A2), we distil the teacher with online DAgger [**?**] into a student that sees only what is available on the robot (o^{prop}, x, g), including the true object pose (position and quaternion), so it is deployable with the motion-capture system rather than the wrist camera. The student executes its own actions in every rollout, perturbed by Gaussian noise with the teacher's learned per-joint standard deviation, and regresses the teacher's mean action with a mean-squared error. After distillation the student loses some performance, so in the third stage (A3) we fine-tune it further with PPO in the same environment with the same reward and recover part of what

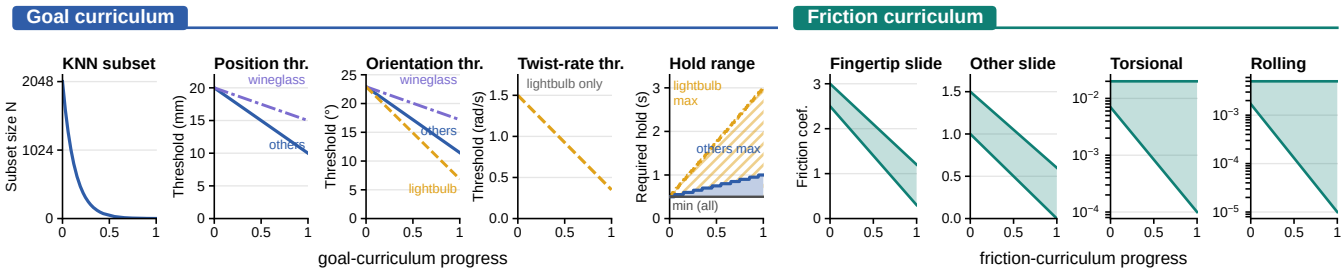


Fig. 19: **Curriculum schedules.** Left: goal-curriculum parameters as a function of its progress p: the KNN candidate subset shrinks from 2048 to 1, so goals move from near the current grasp to uniform over the dataset; the position and orientation tolerances tighten from 2 cm / 0.4 rad to each object's final values; the lightbulb's twist tolerance tightens from 1.5 to 0.35 rad/s; and the maximum required hold rises (band: hold drawn uniformly per goal). Right: friction curriculum: each friction coefficient is sampled per episode from the shaded range, and both bounds move from a grippy start to the final range (torsional and rolling on log axes).

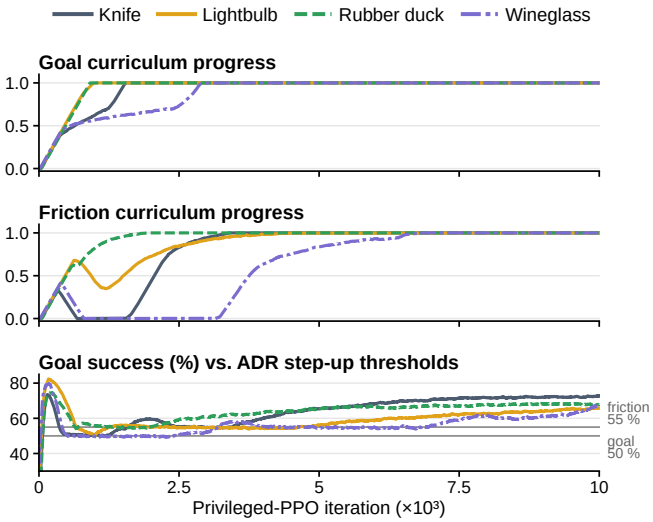


Fig. 20: **Adaptive curriculum progress during A1.** Progress of the goal curriculum (top) and the friction curriculum (middle) over the first 10k iterations for each object, and the rolling goal success that drives them (bottom). Each curriculum steps up while success is above its threshold (goal 50 %, friction 55 %, thin grey lines) and steps down below a slightly lower one.

was lost. A3's actor is initialized from the A2 student and its action standard deviation from A1, and its critic, together with the critic's observation normalizer, is copied from A1, which gives PPO a useful value function from the first iteration. In the final stage of the task policy (A4), we distil the A3 student into a student that replaces the object pose with the wrist depth image. The depth is normalized from $[0.001, 0.35]$ m to $[0, 1]$ and encoded by a CNN followed by a spatial softmax [**?**], whose keypoints and channel means (192 features) are concatenated with the proprioception and the goal and fed to an MLP with LayerNorm, which we found stabilized training. As in DextrAH [**?**], [**?**] and SimToolReal [3], we add an auxiliary head that regresses the current object state from the same features: the position and quaternion, or for axis objects the position, the axis direction and the spin rate. The intuition is that if the encoder can predict the object pose, its features are useful for control. A4 runs at the relaxed success tolerances of 2 cm and 0.4 rad, with distractors and camera randomization on. The end result is two strong policies per object: one that uses the motion-capture system (π^{s}, A3) and one that uses only the wrist camera (π^{d}, A4). These are the state and depth policies of Sec. V, together the *task policies*. All hyperparameters are in Tables X and XI.

Training time. Our runs were spread over several machines (RTX 4090, RTX 4080 SUPER and RTX A6000 GPUs), some of them shared with other jobs, so their logged durations are not comparable. Instead we report each stage's maximum iterations multiplied by the median time per iteration, measured for every stage and object on one machine with RTX 5090 GPUs and run as in training: A1 on two GPUs, every other stage on one. A1 takes about 15 h for 30k iterations (1.8 s per iteration; 28 h for the wineglass, which uses twice as many environments), A2 0.6 h, A3 2.5–2.8 h, A4 10 h, B1 2.5 h and B2 1.8 h. A complete policy and filter for one object thus takes about 33 h (46 h for the wineglass) on this hardware. Each deployed task policy comes from a single training run (one seed per object and observation type).

I. *Safety Filters*

Full description of the safety filter's training (Sec. V-B.2), then the background, the implementation and the comparison of the HJ safety filters (Secs. V-B.3 and VI-B.2).

1) Training the Residual Filter: We also make the correction weight an input. Instead of training one filter per w, we draw $w \sim \mathcal{U}(0, 2)$ for each environment at every episode and give it to both actor and critic (rescaled to $[0, 1]$), so one network covers the whole range and w becomes a dial at deployment: a large w makes corrections expensive and keeps the executed action close to the nominal policy, while $w = 0$ makes corrections free and lets the filter intervene as much as it wants. We found the safety task easier to learn than the goal-reaching task, so, unlike for

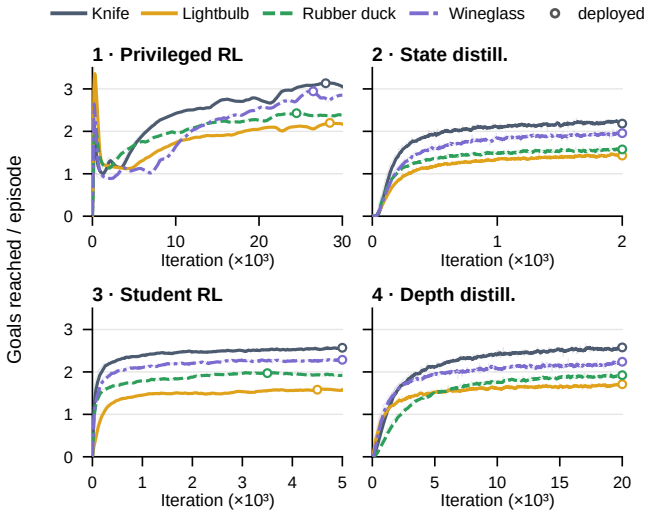


Fig. 21: **Training curves of the four task stages.** Goals reached per episode (EMA) for A1 privileged RL, A2 state distillation, A3 student RL and A4 depth distillation, cut at each stage's budget (30k, 2k, 5k and 20k iterations). Hollow circles mark the deployed checkpoints. Levels are only indicative across stages: observations and success tolerances differ, and A2 rollouts use A1's exploration noise.

the task policy (A1), we use an asymmetric actor–critic whose actor can run on the robot directly. The actor receives only deployable observations, the proprioception and the current object pose (o^{prop}, x) plus a^{nom} and w, and the critic receives the privileged state o^{crit} plus a^{nom} and w. Neither receives any goal observation, so the filter is agnostic to what the task policy is trying to do. For the same reason, we use no curriculum: B1 starts directly at the final curriculum ranges, which is also where the task policy operates. The environment is otherwise the task environment, with the same domain randomization and the goal curriculum pinned at full difficulty, so the nominal policy sees the same goals it gets at deployment. During B1 training only, we perturb the nominal action, $a^{\text{nom}} \leftarrow \text{clip}(a^{\text{nom}} + s\,\epsilon, -1, 1)$ with $s \sim \mathcal{U}(0, 0.5)$ per episode and $\epsilon \sim \mathcal{N}(0, I)$ per step, so the filter also sees worse requests than the nominal's own and is defined off its usual request distribution, e.g., for the depth student or on the real hand. B1 is trained with PPO for 4000 iterations with the frozen state policy A3 as the nominal, and the result is our state safety filter π_δ, used with motion capture. We then distil it with online DAgger (B2, 3000 iterations, same auxiliary pose loss as A4) into a residual student π_δ^{d} that replaces the object pose with the depth image, with the depth policy A4 as the nominal; this is our depth-based safety filter. At deployment we use $w = 0.5$ unless stated otherwise.

2) Hamilton–Jacobi Reachability: Hamilton-Jacobi (HJ) reachability characterizes the largest set of states from which a system can be kept safe for all time [**?**]. Let $l : \mathcal{X} \to \mathbb{R}$ be a margin function over the state space $\mathcal{X}$, whose zero-sublevel set defines the failure set $\mathcal{F}$. For example, l can represent the signed distance between a robot and an obstacle in obstacle-avoidance tasks. The safety value function captures the worst-case margin along a system trajectory $\xi_x^u(t)$ starting at state x under control $u(\cdot)$:

$$V(x) = \sup_{u(\cdot)} \inf_{t \geq 0} l\big(\xi_x^u(t)\big). \tag{11}$$

Thus $V(\cdot)$ characterizes the maximal safe set, defined as $\{x \in \mathcal{X} \mid V(x) \geq 0\}$. In our setting, l is the drop margin ℓ of (2).

In discrete-time, this safety value function can be obtained by solving the discrete-time Hamilton-Jacobi fixed-point equation via dynamic programming:

$$V(x) = \min\Big\{l(x),\ \max_{u \in \mathcal{U}} V\big(f(x,u)\big)\Big\}, \tag{12}$$

where $f(x,u)$ represents the system dynamics. This way the safety of a state is defined as the minimum between the current safety at the state and its future safety under the best control.

[**?**] showed that this safety value function can be approximated using Q-learning and derived the following time-discounted state-action HJ fixed-point equation:

$$Q(x,u) = (1-\gamma)l(x) + \gamma \min\Big\{l(x),\ \max_{u' \in \mathcal{U}} Q\big(f(x,u),u'\big)\Big\}, \tag{13}$$

where $\gamma \in [0,1)$ is a discount factor. In this formulation, the state-action value Q and the state value V are related via the optimal safety fallback policy $\pi_{\text{safe}}(x)$:

$$\begin{aligned} \pi_{\text{safe}}(x) &= \arg\max_{u \in \mathcal{U}} Q(x,u), \\ V(x) &= \max_{u \in \mathcal{U}} Q(x,u) = Q\big(x, \pi_{\text{safe}}(x)\big). \end{aligned} \tag{14}$$

By leveraging the learned $Q(x,u)$, the safe fallback policy $\pi_{\text{safe}}(x)$, and a given nominal task policy $\pi_{\text{task}}(x)$, one can construct a least-restrictive safety filter (LRSF). When a safety violation is imminent, the filter abruptly intervenes by running the safe fallback action:

$$u(x) = \begin{cases} \pi_{\text{task}}(x), & \text{if } Q\big(x, \pi_{\text{task}}(x)\big) > 0, \\ \pi_{\text{safe}}(x), & \text{otherwise.} \end{cases} \tag{15}$$

3) Smooth Safety Filtering with Q-CBFs: An issue with LRSF is that its hard switching mechanism can result in sudden, undesirable control interventions. One solution to mitigate this is the use of control barrier functions (CBFs), which admit a smooth control law. Following [**?**], [**?**], the safety value function inherently satisfies the conditions of a discrete-time control barrier function (DCBF) [**?**].

Leveraging this property, the Human-Centered Safety Filter (HCSF) [**?**], [**?**] avoids hard switches by minimally steering the nominal policy $\pi_{\text{task}}(x)$ to ensure safety. At runtime, it solves the following optimization problem:

$$\begin{aligned} u(x) = \arg\min_{u \in \mathcal{U}} \quad & \|u - \pi_{\text{task}}(x)\|^2 \\ \text{s.t.} \quad & Q(x,u) \geq \beta\, Q\big(x, \pi_{\text{safe}}(x)\big), \end{aligned} \tag{16}$$

where $\beta \in (0,1]$ dictates how aggressively the filter overrides the nominal policy as the system approaches the safety boundary. We refer to the constraint as the CBF constraint.

Because solving this optimization is intractable when Q is parameterized by a neural network, it is approximated in practice using a line-search sampling-based approach. By evaluating candidate actions interpolated between the nominal input $\pi_{\text{task}}(x)$ and the safe fallback action $\pi_{\text{safe}}(x)$, the filter can select the action closest to the nominal input while satisfying the CBF constraint.

4) Implementation: These filters were introduced with SAC [**?**], which is not designed for thousands of parallel simulated environments, so we use the same idea on FlashSAC [**?**] and change only its Bellman target to the safety backup (13): $y = (1-\gamma)\,l(x) + \gamma \min\{l(x), Q'\}$ with $\gamma = 0.997$, 3-step returns, and $y = l(x)$ at a true termination. FlashSAC's critic is distributional (101 atoms on $[-1, 1]$). Instead of using the mean as the safety value, we gate on its conditional value at risk CVaR_α, the mean of the worst α fraction of the distribution, which makes the estimate more conservative and the gate trigger more often; the fallback actor still maximizes the mean. LRSF switches to the fallback when $\text{CVaR}_\alpha(Q(x, u_{\text{task}})) \le \epsilon = 10^{-3}$. HCSF approximates (16) with a simple zeroth-order search, evaluating 16 candidates on the segment from the nominal action to the fallback action and executing the one closest to the nominal that satisfies the CBF constraint on CVaR, falling back to π_{safe} if none does. We train the HJ filter on the rubber duck, in the same environment as ResSafe, with 1,024 environments for 50k iterations (actor MLP 2×128, critic 3×512). During data collection the filter runs behind the nominal policy, half of the environments use the LRSF gate and half the HCSF gate with α and β drawn per episode, and the nominal action is perturbed as for ResSafe. We sweep $\alpha \in \{0.05, 0.1, 0.2, 0.3, 0.5\}$ and $\beta \in \{0.5, 0.75, 0.9, 0.95, 0.99\}$ at evaluation. Since both the HJ actor and critic run at deployment (the critic gates every action), both read the same deployable, goal-free observation as ResSafe's actor: the joint history and the noisy, delayed object pose, without velocities or other privileged simulator state. We train three seeds and report the best. The training budgets differ, as is usual between off-policy and on-policy RL: each HJ run collects 51M transitions (1,024 environments × 50k iterations of one step each) with two gradient updates per iteration and takes about 1 h on one RTX 4080 SUPER, whereas B1 collects 786M transitions (8,192 environments × 24 steps × 4,000 iterations) with 80k mini-batch updates and takes about 3.9 h on the same GPU. We also train the HJ filter for 180k iterations (3.6×, about 3.9 h on the same GPU, matching B1), all else identical. Its fraction of unsafe training episodes stops improving after about 100k iterations, while its critic keeps growing more pessimistic, so at matched drop rates it reaches 4–9 points fewer precise goals and changes the action more (Table XII); we therefore report the 50k filter. Table XIII lists every swept setting of it.

5) Comparison with the HJ Filters: Table I compares ResSafe with the HJ filters of Sec. I.2 on the rubber duck with the state policy. For a fair comparison, the HJ filter's actor and critic read the same deployable, goal-free observation as ResSafe's actor (no velocities or other privileged simulator state), so it could run on the real robot. ResSafe at $w = 0.5$ halves drops (from 9.1 to 4.5 %) while reaching 91.3 % of goals (coarse), within one point of the unfiltered policy. From the full sweep over $\alpha \in \{0.05, \ldots, 0.5\}$ and $\beta \in \{0.5, \ldots, 0.99\}$, the table shows the HJ settings closest to ResSafe on either side of its drop rate, and each is worse. HCSF at $\alpha = 0.3$, $\beta = 0.9$ comes closest in success (88.2 % coarse, 79.8 % precise) but drops the object more often (5.9 %) and sooner (199 vs. 266 s to the first drop). At ResSafe's drop rate (HCSF at $\alpha = 0.2$, $\beta = 0.9$: 4.6 %), it reaches 83.1 % of goals (72.1 % precise), 8 points below ResSafe on both, while changing the nominal action slightly more (15.8 vs. 15.3 %). Pushing drops just below ResSafe's ($\alpha = 0.3$, $\beta = 0.95$: 3.9 %) costs 10 points of coarse and 11 points of precise success, and the hard-switching LRSF at its best $\alpha = 0.1$ (larger α barely reduces drops) reaches only 74.6 % of goals with more drops (6.5 %). The HJ filter can push drops lower (0.3 % at $\alpha = 0.1$, $\beta = 0.99$), but only by blocking most goals (10.5 % reached). It also needs a runtime search over candidate actions, and is sensitive to the choice of α and β, whereas ResSafe is a single forward pass whose trade-off is set by one input. For both filters we report the best of three training seeds. The HJ settings α and β were selected on this same evaluation, while ResSafe's $w = 0.5$ was fixed beforehand, which if anything favors the HJ filter.

J. Real-Robot Evaluation

Table XIV summarizes the protocol of every real-robot experiment, Fig. 22 shows the outcome of every goal, and Fig. 7 the six wrist orientations. The full protocol of the main experiment (Sec. VI-C.1):

Our first set of experiments tests performance, reliability and safety by running long sequences of goals at different wrist orientations. We choose the six wrist orientations of Fig. 7, which point gravity along each of the six axis directions of the hand: palm up, palm down, and palm sideways with the fingers pointing forward, up, backward and down. We test the state policy A3 with and without the state safety filter B1, and the depth policy A4 with and without the depth filter B2, both filters at $w = 0.5$. We use motion capture in every trial, as the object-pose input of the state policies and as ground truth for measuring the position and orientation errors of all policies. We use the same evaluation criterion as in simulation: a goal is reached if the object is within the coarse (2 cm, 0.4 rad) or precise (1 cm, 0.2 rad) tolerance at any point in its 10 s window. The goals come from a separate evaluation grasp dataset of 100,000 grasps per object, generated with the same procedure as the training dataset (20,000 LightningGrasp seeds expanded by RRT) but with a different random seed, so the goals are not seen in training; this set is not penetration-filtered. For each object and wrist orientation, we draw a sequence of 50 goals with a fixed seed, so every variant receives exactly the same goal sequence, and run the 50 goals consecutively. To keep the experiment consistent, we do not stop a run when the object is dropped. Instead, when the object is more than 25 cm from

the hand for three consecutive motion-capture frames, the hand opens to a fixed starting grasp, we place the object back in the hand, and the run continues with the next goal. If the hand loses its connection mid-goal, for example when the object hits the hand's USB cable, we reset the hand to the same starting grasp and redo that goal window, which is not counted. If the object gets stuck, i.e., the hand reaches a configuration it cannot recover from, we use the same reset: when two consecutive goals fail with the object nearly still (moving less than 10 mm and 10° in the last 5 s, with its best error more than twice the coarse tolerance), the hand is reset and the run continues with the next goal, without redoing it. The lightbulb can also be commanded to spin about its axis, but here we keep its twist command at zero, so it is evaluated like the other objects (we evaluate spinning separately below). For each run we count the goals reached at the coarse and precise tolerances and the drops. Each variant thus attempts 300 goals per object (6 orientations $\times$ 50 goals), and the 4 variants $\times$ 4 objects total 4,800 goals and 13.3 hours of goal windows (16.4 hours including resets), reported in Table II and Fig. 22. With the wide-goal and lightbulb-twist tests below (400 goals of 10 s and 200 of 20 s), the real-robot evaluation totals 5,400 goals and 15.6 hours of goal windows.

1) Results of the Main Experiment: Across objects, success orders lightbulb > rubber duck > knife > wineglass for both the state and the depth policy. Without the filter, the state policy reaches 95.2 % of all goals at the coarse tolerance (95 % confidence interval 93.8–96.2 %) and 75.6 % at the precise one (73.1–77.9 %); with the filter ($w = 0.5$), coarse success drops to 91.5 %. The wineglass in particular does much worse at the precise tolerance (141 of 300 goals). We attribute this to its training: its final tolerances were loosened to 1.5 cm and 0.3 rad, so it was never trained to reach the precise tolerance. Overall, the state policies match what we expected from simulation: coarse success is similar to simulation (95–99 % vs. 92–95 % for knife, lightbulb and duck; 89 % for the wineglass in both), and drops are, if anything, rarer on the real robot (0.7–2.3 % of goals vs. 3.7–9.1 % in simulation), although the two protocols differ: the simulation benchmark uses random wrist orientations, goals from the training distribution and the 0.3 m safe set, while the real runs use six axis-aligned wrist orientations, held-out goals and a 25 cm drop distance. For the depth policy, the lightbulb and rubber duck do about as well as in simulation (95 and 88 % coarse success), but the knife and wineglass do much worse in both performance and safety (54 and 50 % coarse success, against 90 and 84 % in simulation); the wineglass in particular drops very often (51 of 300 goals).

Besides goal success at our two tolerances, we measure safety, and safety together with performance: the number of drops, the time before the first drop (or the end of the 500 s run), and the number of goals reached before the first drop (all goals reached in the run if it never drops), each averaged over the 6 runs per object. On a goal-by-goal basis, the filter costs performance: with the state policy, coarse success drops from 95.2 to 91.5 % and precise success from 75.6 to 67.2 %; with the depth policy, from 71.8 to 66.7 % and from 38.3 to 31.8 %. This is expected: the filter is not aware of the goal and was never rewarded for reaching it. However, when we consider safety, and how safety ties into performance, the filter gives large gains. With the state policy, drops fall from 16 to 4 (of 1,200 goals), the mean time to the first drop rises from 334 to 465 s, and the mean number of goals reached before the first drop rises from 31.8 to 42.9 (of 50). With the depth policy, drops fall from 88 to 46, the time to the first drop rises from 228 to 354 s, and goals reached before the first drop rise from 17.9 to 26.0. Both drop reductions are statistically significant (two-sided exact test on how the drops split between the two conditions, which ran the same 1,200 goals: $p = 0.012$ for the state policy and $p < 0.001$ for the depth policy), although per object the counts are too small to be significant on their own. In other words, with the filter the robot reaches more goals before a catastrophic drop, and lasts longer before dropping, even though it reaches slightly fewer goals overall. Fig. 24 shows this per object, and Figs. 8 and 25 show drops that the filter prevented. The wineglass with the depth policy is the exception: its drops only fall from 51 to 40, consistent with the visual gap discussed above.

Full description of the out-of-distribution experiment (Sec. VI-C.3; Fig. 23, Table XV):

To stress-test the system, we also evaluate very difficult goals (Table XV, Fig. 10). We create another evaluation dataset of 100,000 grasps per object with the same pipeline (20,000 LightningGrasp seeds expanded by RRT, with a different random seed), but with the canonical box enlarged 1.5$\times$ about its center, so the object can be placed farther from the palm. As before, the grasps pass the generator's force-balance and collision tests but are not physically validated in simulation or on the robot, so some goals may simply be unreachable. Fig. 23 visualizes, for the knife, the goals of one run of this wide dataset next to the goals of the main experiment. Of the goals we ran, only 40–56 % of the wide goals lie inside the original box, against 80–88 % of the main evaluation goals; over the full datasets, 35–44 % of the wide grasps do, against 80–86 % of the training grasps, so these goals lie largely outside the training distribution. We test the state policy with and without the filter, with the palm down, one of the harder wrist orientations. Surprisingly, the harder goals do not seem to hurt the safety of the nominal task policy: only the wineglass dropped, twice with and twice without the filter, which is likely too few trials to measure the filter's effect. They certainly affect success: the policy has difficulty reaching many of the goals that are far from the hand or in strange, hard-to-grasp orientations, so coarse success drops to 28–42 of 50 goals (56–84 %), well below the main experiment. With the filter, success drops further (19–39), presumably because the filter is more conservative about reaching to these places. Fig. 10 shows some of the very difficult goals that the policy reached, such as holding the knife by the very tip of its handle or holding the lightbulb without the thumb. They show how far we can push the system: it remains remarkably capable, while still managing safety: rather than risk a drop, it tends not to attempt goals



Fig. 22: **Outcome of every goal in the real-robot evaluation.** Columns are the state (motion-capture) and depth policies; rows are without and with the safety filter. Each row of cells is one run (4 objects × 6 wrist orientations) and each cell one of its 50 consecutive goals. Dark blue: reached at the precise tolerance (1 cm / 0.2 rad); light blue: coarse only (2 cm / 0.4 rad); grey: not reached; orange: drop. The filter reduces drops from 16 to 4 (state) and from 88 to 46 (depth).

that are close to impossible.

K. Additional Results

Further results of Sec. VI, with the full descriptions of the experiments condensed in the main text.

1) Where Does the Depth Gap Come From?: Since the state policies transfer well, the simulated dynamics seem adequate, and the depth observation is the only change. To test whether the gap is perceptual, we replay recorded real wrist stereo pairs through the stereo network and the depth policy, read out its auxiliary object-pose head, which predicts the object pose from the same features the policy acts on, and compare it with motion capture (Table XVII, Fig. 26). We do this for two sets of real runs: those of the state policy, in which the object mostly stays in nominal, goal-like poses, and those of the depth policy itself. In the state runs, the head is about as accurate on the real robot as in simulation for the rubber duck, the lightbulb and the knife (knife: median orientation error 13 against 10°, errors above 45° in 8 against 5 % of the steps), but not for the wineglass: its median position error grows from 10 to 15 mm, and its tilt error exceeds 45° in 18 % of the steps against none in simulation. In the depth runs, the knife is also misperceived (median 24°, 37 % of the steps above 45°, mostly flips), but almost only in goals that fail or end in a drop (14° in reached goals against 104–114° in the others). In the real depth images, the knife blade and the bowl of the wineglass appear as only a sliver of valid pixels. The wineglass is thus perceived worse on the real robot even in nominal poses, which points to the stereo depth of its thin, curved surfaces as a main cause of its gap. The knife is perceived well in nominal poses and badly in the unusual poses that its failures produce, so for the knife we cannot tell whether misperception causes the failures or follows from them.

2) Spinning the Lightbulb: We also evaluate how well the lightbulb can be spun about its own axis. We test this with the depth policy: the bulb is symmetric about its z-axis, so its exact pose cannot be distinguished from vision, and vision-based policies typically do not control both rotation and position. With our axis goal and target twist, the policy can move the bulb to a target position, point its axis in a target direction, and then rotate it about that axis (Fig. 9). We test palm up, the easier configuration, and palm down, where the policy finds it harder to rotate the bulb. Each run has 50 goals of 20 s from the evaluation dataset of the main experiment, each with a twist command of $+1$ or -1 rad/s chosen at

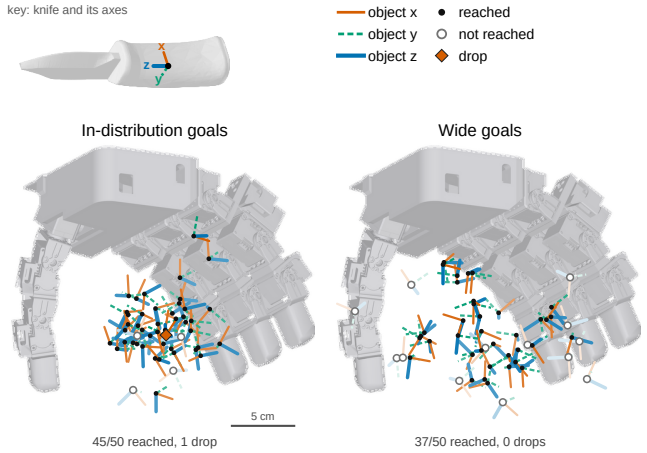


Fig. 23: **Goal diversity, knife, palm down.** Every goal of one real run is drawn as the object frame at its goal pose relative to the hand (x, y, z axes; key at top), for the main evaluation goals (left) and the wide goals from a 1.5× larger grasp box (right). Markers show the outcome without the filter: reached (dot), not reached (open circle, faded triad) or dropped (diamond). Outcomes: 45/50 reached with 1 drop, and 37/50 reached with 0 drops.

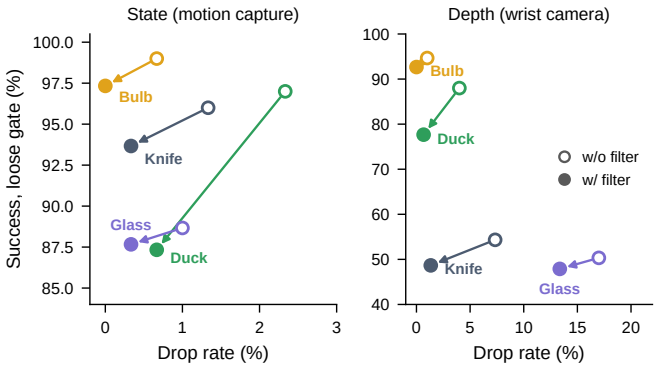


Fig. 24: **Goal success versus drop rate on the real robot.** Per object, an arrow goes from the result without the safety filter (open marker) to the result with it (filled marker), for the state (motion-capture) policy (left) and the depth policy (right). Each point pools 6 wrist orientations × 50 goals; success uses the coarse tolerance (2 cm / 0.4 rad). Note the different axis ranges.

random. We record whether the target position and axis are reached at the coarse tolerance and count how many turns the bulb makes about its axis in the commanded direction while within that tolerance, as well as drops. Without the filter, the policy reaches 46 and 44 of 50 goals (palm up and down) and makes at least one full turn in 28 and 12 goals, with a mean of 1.24 and 0.63 turns per goal (Table XVIII). Again, the safety filter (the same filter and policy as in the main experiment) costs some performance, both in reaching the goal (43 and 41) and in turning (0.74 and 0.36 turns per goal), and since the lightbulb rarely drops, there were no drops in any run. The point is that the policy is very flexible and can hold the lightbulb at diverse positions and orientations while spinning it. Wrist orientation and the in-hand goal do matter: at least one full turn was fairly rare with the palm down, and at some lightbulb poses it seems extremely difficult to rotate the bulb at all, which may be a hardware limitation as much as a policy one. We also notice that when the hand is forced to do these harder things, the nominal policy does not become less safe. We believe this is a property of how the nominal policy is trained: it is rewarded over a horizon of consecutive goals and penalized for dropping, so it does not push itself on motions it deems too risky.

3) Unseen Arm Motion: We next introduce out-of-distribution physical effects and evaluate safety under them. We evaluate the depth policies, since this test cannot run inside the motion-capture cage while the arm moves this much. The wrist orientation now changes while the policy runs, so gravity rotates in the hand frame, and the object is subject to inertial forces as the hand is swung through the trajectory, neither of which occurs in training. The hand rotates an object in place: the lightbulb is given a goal and a twist command to spin it (-1.5 rad/s, or -1 rad/s for the high goal), and for the rubber duck, whose goal is a full pose, we continually rotate the goal about the duck's z-axis at 0.5 rad/s. After 5 s of settling, the arm moves the hand for 30 s with a sinusoid on each of the three UR5 wrist joints (0.15–0.27 Hz, amplitudes up to 85°, peak angular speed 2.75 rad/s), which changes the hand's orientation very quickly and sweeps gravity through most directions. We run 10 trials per setting, at two goal heights above the palm (the higher one at the top of each object's goal range, which is harder), and count the trials in which the object is still in hand at the end (Table XIX, Fig. 11). Without the filter, 32 of 40 trials hold on; with the filter, 38 of 40, with the largest gains at the higher goals (7/10 to 10/10 for the lightbulb, 7/10 to 9/10 for the duck). Qualitatively, at the 15.6 cm lightbulb goal the filter sometimes keeps the bulb but stops rotating it: in 3 of the 10 trials the fingers barely moved. We suspect that in these hand configurations the filter corrects strongly enough to stall the policy: the bulb stays safe but does not rotate. More importantly, the policy keeps spinning the objects under large disturbances it never saw in training, and the safety filter makes it safer still.

4) Qualitative Robustness: Finally, we run qualitative tests of the robustness of the goal-reaching policy with the depth policies of the rubber duck and lightbulb, without the filter (Fig. 12). A person pulls fingers away from the object (first the ring finger, then the middle finger), and the remaining fingers keep the object near the goal; the lightbulb tips by about 30° and is righted. A person also pulls the object into the palm, and after release the policy regrasps it and returns it to the goal within a few seconds. A hand or arm moving behind the hand, or briefly covering the camera, does not disturb the policy either. None of these perturbations were experienced in training.

5) Downstream Tasks: We then test downstream tasks that we design to showcase combining our in-hand goal-reaching policy with other actions, in our case hard-coded

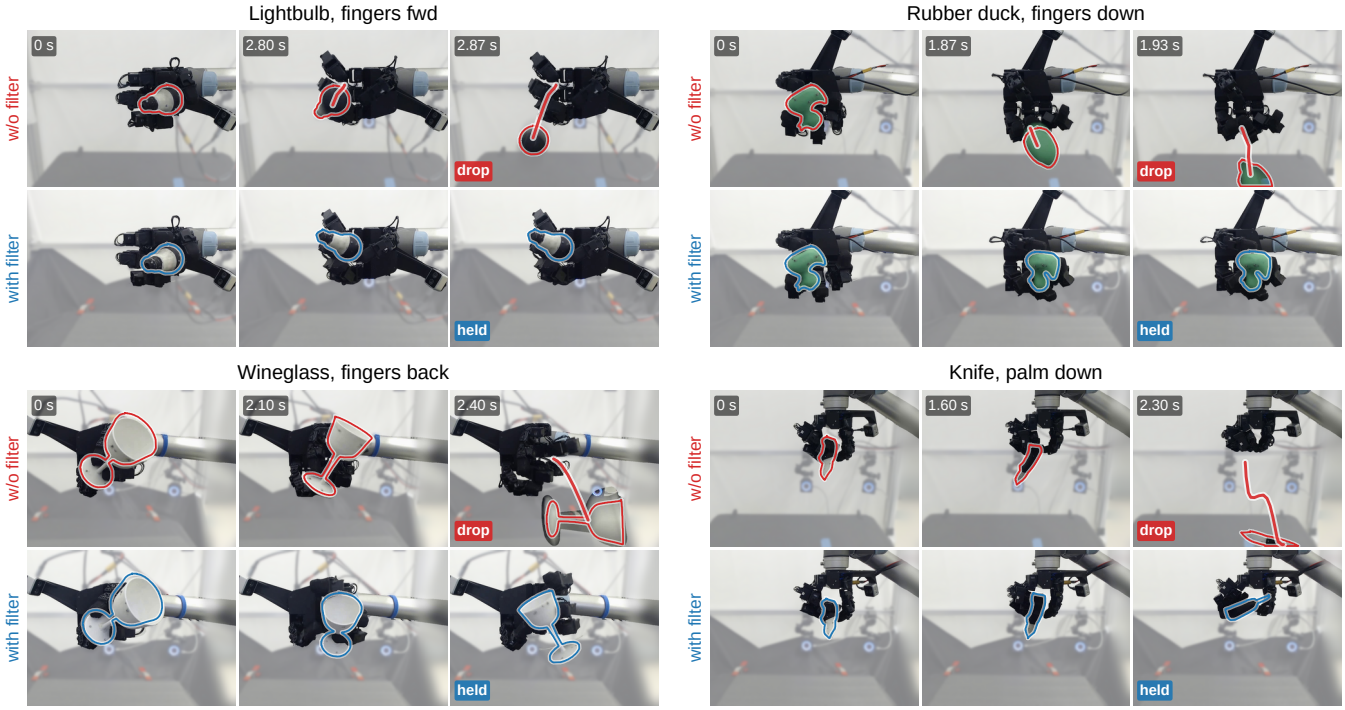


Fig. 25: **More drops prevented by ResSafe.** Each block shows the same goal attempted without (top) and with (bottom) the filter: lightbulb fingers forward, rubber duck fingers down, wineglass fingers back, and knife palm down. Frames are at goal onset, just before the slip, and once the object has fallen. Outlines show the tracked object pose, the red line traces the measured fall path from slip onset, and times are from goal onset.

arm motions (Fig. 13, Table XX). We run the depth policy of the object being manipulated, and we run them without the lab setup: without the motion-capture system, on objects without markers, with the robot arm simply placed next to an ordinary table. In the wineglass task, the hand grasps and picks up the glass, flips it 180° in hand so that the cup points down, and flips it back; the flips are done by the hand alone while the arm holds still. In the lightbulb task, the hand starts with the bulb lying in the palm, regrasps it upright, carries it under a socket, screws it in with a continuous twist goal, and the bulb lights up. In the rubber duck task, the hand grasps the duck lying on its side on the table, picks it up, reorients it upright, and places it back on the table upright. The goals for each stage are hand-picked in-hand poses. These tasks involve many situations the policy never saw in training. It must grasp and pick up objects from a table: we only open the hand into a rough pregrasp near the object and start the policy, and it closes into a grasp. In the screwing task, once the bulb catches the socket thread, part of it is fixed to the world, which the hand has never experienced. The wineglass task succeeds in 7 of 10 trials without the filter and 6 of 10 with it, the lightbulb lights up in 4 of 5 trials without and 3 of 5 with the filter, and the duck is placed upright in 7 of 10 trials without and 3 of 10 with the filter. These tasks introduce much more that can go wrong besides the in-hand policy, such as the initial object placement and the hard-coded arm and pregrasp motions, and we ran a limited number of trials to showcase the ability, so the success rates are not the main point. We run the filter at $w = 1.5$ here, since at the deployed $w = 0.5$ it was too conservative for these tasks; even so, it does not do as well. This could have several causes: the filter may handle the out-of-distribution parts of the tasks less well, and the hard-coded parts (arm motions, pregrasp and placing) add randomness to the outcomes. Either way, the policy works with only the wrist camera, with clutter in the background in the depth image (Fig. 14). In the future, such general goal-reaching skills could be combined with higher-level planners or learned high-level policies, such as vision-language-action models, to obtain more general policies for non-anthropomorphic hands that do not have five fingers, and still reach human or superhuman dexterity.

6) Learning a New Constraint: The filter is not specific to drops: the same recipe learns other constraints by changing only the safety margin. As a demonstration, we train a filter for the knife that must also keep the hand off the blade. Its margin is the minimum of the drop margin (2) and a normalized clearance between the hand's collision geometry and the blade, computed at every control step, which is violated when any part of the hand comes within 1 mm of the blade. The reward, observations, nominal policies and training are those of B1 and B2, unchanged, except that episodes start only from grasps that satisfy the constraint, since an episode that starts in violation ends at once and teaches nothing. We distil the filter to depth on top of the knife depth policy and run it on the real robot at $w = 2$,

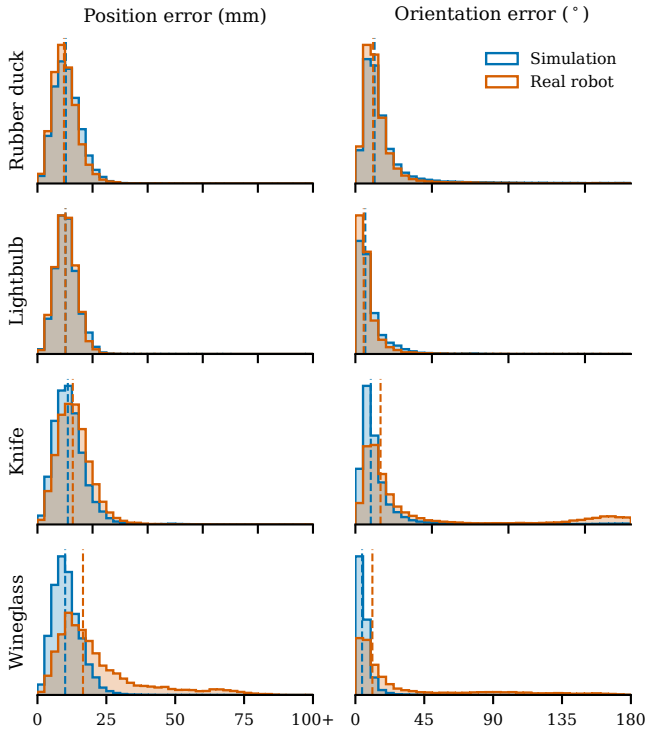


Fig. 26: **Object pose perceived by the depth policy.** Distribution of the error of its auxiliary pose head against the true object pose over in-hand control steps, in simulation (blue) and on the real robot (orange). The real errors pool every run of the main experiment, replayed from its recorded wrist stereo pairs: state- and depth-policy runs, with and without the filter (Table XVII splits them). Dashed lines are medians; orientation is the full rotation for the rubber duck and knife and the axis tilt for the lightbulb and wineglass; position errors above 100 mm (4 of about 180,000 steps) are collected in the last bin. The duck and lightbulb are perceived as well as in simulation. The knife's orientation sometimes flips by nearly 180°, almost only in the depth-policy runs, and the wineglass's position and tilt errors grow a long tail.

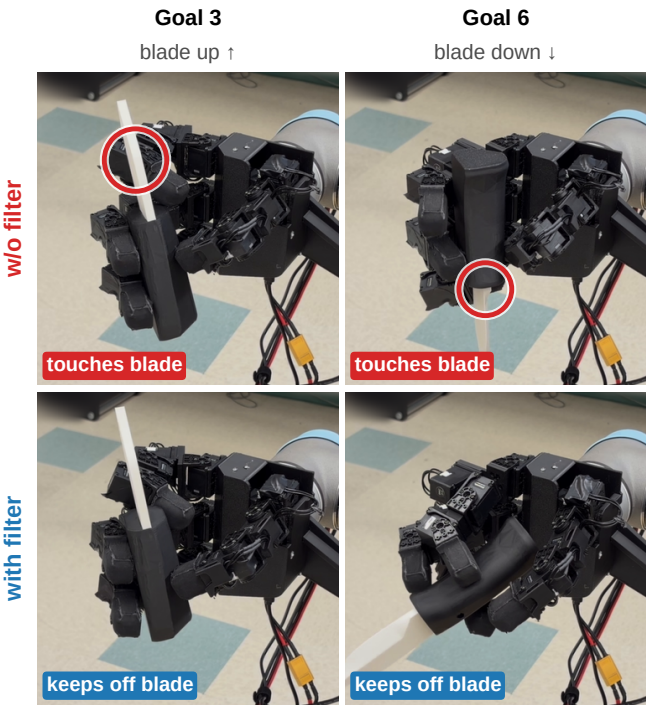


Fig. 27: **Learning a new constraint: keep the fingers off the knife blade.** The same goal (columns: goal 3, blade up; goal 6, blade down) without the filter (top), where a finger touches the blade (red ring, placed by eye), and with a residual filter trained to avoid blade contact, run at $w = 2$ (bottom), which reaches the goal with the fingers off the blade. The nominal policy is the knife depth policy (A4).

three times each with and without the filter, on a sequence of seven goals that flip the knife between blade up and blade down (Fig. 27). Without the filter, a finger touches the blade at several goals; with it, the hand reaches the same goals with its fingers off the blade. We evaluated this qualitatively only.

7) Curriculum Ablation: We compare several curriculum variants on A1 (Fig. 28, Fig. 6). The friction curriculum contributes the most and learns fastest. On the knife, over 3 seeds and a 10k-iteration budget, goal + friction reaches 2.27 goals per episode, friction only 2.14, no curriculum 1.87 and goal only 1.33. On the lightbulb and rubber duck (one seed), the friction curriculum gives 15–64 % more goals than no curriculum. On the wineglass, however, no curriculum ends highest at this budget (2.74 vs. 2.20 goals per episode, single seed), so the benefit is not universal. We also test a gravity curriculum, variants of which are used in TeleDexter [6] and Visual Dexterity [2], which ramps gravity from 0 to 9.81 m/s^2 together with the disturbance forces. In our setting it does not help (1.11–1.14 goals per episode on the knife), and most gravity runs never reached full gravity within the budget. Our guess is that the grasp resets and the diverse wrist orientations already make the early task learnable. The goal curriculum alone does not help either, since it delays reaching full difficulty, but we keep it together with the friction curriculum for our experiments, which was the best combination on three of four objects.

8) ResSafe Design Ablation: We ablate two design choices of the filter on the rubber duck, with 3 training seeds each (Fig. 29, Fig. 31). First, the norm of the correction penalty: with an L1 penalty ($\|\delta\|_1/n$ instead of $\|\delta\|_2^2/n$), the filter becomes all-or-nothing. At $w = 0.5$ it reaches only 21.7 % of goals (coarse) against 80.4 % for L2, at 1.3 % vs. 3.8 % drops, and at $w \leq 0.5$ it essentially takes over and stops the task. The quadratic penalty makes small corrections cheap and large ones expensive, which produces a usable dial. Second, the noise added to the nominal action during training: without it, results are within the seed spread of the final filter (78.6 vs. 80.4 % coarse success and 2.5 vs. 3.8 % drops at $w = 0.5$), so it does not improve the simulation metrics. We keep it because it exposes the filter to requests that are worse than the nominal policy's own, which we

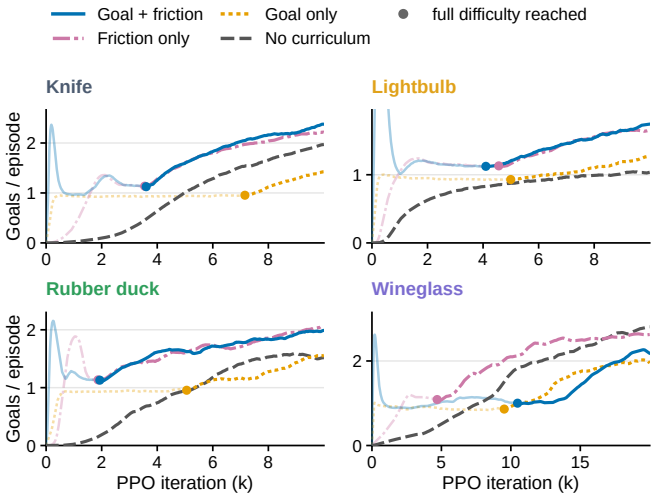


Fig. 28: **Curriculum ablation for privileged training (A1).** Goals reached per episode for goal + friction curricula, friction only, goal only, and no curriculum (one seed, 10k iterations; 20k for the wineglass). Curves are faded while a curriculum is still ramping (an easier task), and the dot marks full difficulty. The friction curriculum helps the knife, lightbulb and duck (+15 to +64 % over no curriculum at the end of training); the goal curriculum alone does not help at this budget; for the wineglass, no curriculum ends highest.

expect to matter when the filter is distilled to depth and paired with a different nominal policy (A4) and on the real hand.

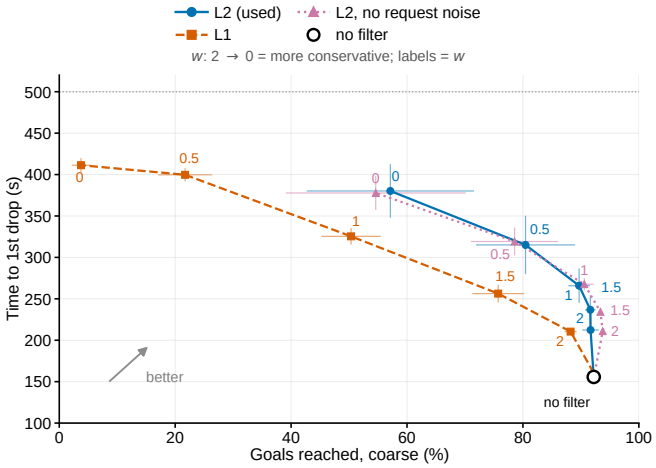


Fig. 29: **ResSafe penalty ablation (rubber duck, simulation).** Coarse goal success versus time to the first drop for the L2 correction penalty (ours), an L1 penalty, and L2 without nominal-action noise during training. All share the no-filter point; labels give w. Mean $\pm$ s.e.m. over 3 seeds. The L1 penalty buys drop-free time by collapsing goal success (21.7 % coarse at $w = 0.5$ vs. 80.4 % for L2), while L2 with and without the noise overlap within the seed spread.

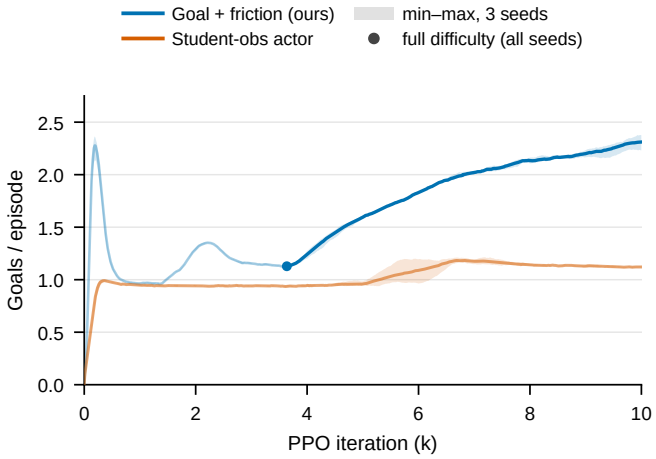


Fig. 30: **A1 needs privileged actor observations.** Knife goals per episode with the privileged actor (ours) versus an actor that sees only the student's noisy, delayed observations (o^{prop}, x, g), with the same critic and curricula. Mean with min–max band over 3 seeds. The student-observation actor plateaus at 1.12 goals per episode versus 2.27, with no seed overlap; its curricula also finish much later (goal curriculum at 5–6k iterations instead of about 1.5k).

TABLE VIII: Domain randomization of the simulated environment, shared by every training stage (Fig. 4). Actuator randomization is in Table V. $\mathcal{U}$, $\log\mathcal{U}$: uniform, log-uniform; $\pm x$: $\mathcal{U}(-x, x)$. Drawn once per environment (env), per episode (ep.) or per control step (step, 20 Hz). Privileged inputs are noise- and latency-free.

Quantity	Range or distribution	Drawn
Contact friction (start of A1 curriculum → final range, used throughout A2–A4, B1–B2)		
Fingertip sliding μ	$\mathcal{U}(2.5, 3.0) \rightarrow \mathcal{U}(0.3, 1.2)$	ep.
Non-fingertip sliding μ	$\mathcal{U}(1.0, 1.5) \rightarrow \mathcal{U}(0, 0.6)$	ep.
Torsional μ	$\log\mathcal{U}(7, 20) \rightarrow \log\mathcal{U}(0.1, 20)\,(\times 10^{-3})$	ep.
Rolling μ	$\log\mathcal{U}(1.7, 5) \rightarrow \log\mathcal{U}(0.01, 5)\,(\times 10^{-3})$	ep.
Inertial[a]		
Robot link mass & inertia	$\times e^{2\alpha}$, $\alpha \sim \mathcal{U}(-0.15, 0.15)$	env
Robot link CoM	$\pm$1 mm per axis	env
Object mass	$\log\mathcal{U}$ over the object's range (Tab. VI)	env
Object CoM	$\pm$10 % of bounding box per axis	env
Object inertia stretch	$\mathcal{U}(-0.05, 0.05)$	env
Geometry		
Robot link frames	$\pm$0.3 mm, $\pm$0.01 rad per axis	env
Robot colliders	$\pm$0.5 mm, $\pm$0.01 rad; box size $\times\mathcal{U}(0.97, 1.02)$	env
Object colliders	$\pm$0.5 mm per axis	env
Perturbations and initial state		
Object disturbance force[b]	$2m\,\hat{\mathbf{u}}$ N, new w.p. 0.2 per step, $\times$0.9 per 80 ms	step
Wrist orientation	uniform on $SO(3)$	ep.
Initial grasp	uniform over the object's grasp set	ep.
Proprioception o^{prop} (A2–A4, B1–B2)		
Joint position noise	$\mathcal{U}(-0.005, 0.005)$ (normalized to $[-1, 1]$)	step
Command residual noise	$\mathcal{U}(-0.005, 0.005)$ rad	step
Joint position latency[c]	$\mathcal{U}\{0, 1\}$ control steps (0–50 ms)	step
Command residual latency[c]	$\mathcal{U}\{0, 1\}$ control steps	step
Projected gravity latency[c]	$\mathcal{U}\{0, 1\}$ control steps	step
Object pose x (A2, A3, B1)		
Position noise	$\pm$3 mm per axis	step
Orientation noise	$\pm$0.035 rad per axis (rotation vector)	step
Latency[c]	$\mathcal{U}\{0, 1, 2\}$ control steps (0–100 ms)	step
Depth camera I (A4, B2)[d]		
Camera position	$\pm$3 mm per axis	ep.
Camera orientation	$\pm$0.03 rad per axis	ep.
Field of view	$\times\mathcal{U}(0.98, 1.02)$	ep.
Image latency[c]	$\mathcal{U}\{0, 1, 2\}$ control steps (0–100 ms)	step
Pixel jitter	$\mathcal{N}(0, 0.2^2)$ px, bilinear resample	step
Disparity noise	$\mathcal{N}(0, 0.025^2)$, then quantized	step
Surface-normal noise	$\sigma_\theta = 20$	step
Pixel dropout	$p = 0.001$ (set to 0)	step
Random-value pixels	$p = 0.001$	step
Stick artifacts	$p = 5\times10^{-4}$, $\leq$10 px long, $\leq$2 px wide	step
Distractor objects	6 primitives, each w.p. 0.6, at 6–32 cm, drifting	ep.

[a] Mass, inertia and CoM are perturbed jointly through the pseudo-inertia matrix [**?**], which keeps the mass positive and the inertia tensor positive definite. [b] Force only, applied at the CoM in a uniformly random direction $\hat{\mathbf{u}}$, following [**?**]. [c] Redrawn every control step but kept with probability 0.8, so it behaves as a latency rather than reordering frames. [d] 160×120, 65.8° vertical FoV; returns outside [0.001, 0.35] m are set to 0. Noise model as in [**?**]. Distractors are visual only (slab, box, capsule, cylinder, sphere, ellipsoid) with random size ($\times\mathcal{U}(0.5, 2)$ per axis) and orientation, and never occlude the object.

TABLE IX: Observations, grouped by the symbols of Fig. 4. Dim: per-frame size; Hist.: frames stacked per term (width = Dim × Hist.). Noise and latency on the deployable inputs (o^{prop}, x, I) are in Table VIII; privileged inputs are clean and undelayed. Positions, velocities and poses are in the hand frame. Depth students also regress x as an auxiliary target ($\hat{x}$), which is not an input.

Term	Dim	Hist.
o^{prop} *proprioception*		
Projected gravity	3	1
Last action	16	1
Command residual $(q^{\text{cmd}} - q)/0.1$	16	1
Joint position[a]	16	10
x *object pose (motion capture)*		
Object position	3	1
Object orientation (quaternion)	4	1
Goal (no ResSafe network reads any goal term)		
g *goal*		
Goal pose in hand frame[b]	7	1
g^{priv} *privileged goal*		
g	7	–
Goal relative to object	7	1
g^{crit} *critic goal*		
g^{priv}	14	–
Position / orientation error	2	1
In-gate flag	1	1
Hold progress	1	1
Required hold length	1	1
Twist error	3	1
I *wrist depth*		
Depth image (Table VIII)	120×160	1
o^{priv} *privileged state*		
Projected gravity	3	1
Last action	16	1
Command residual	16	1
Joint position (true angle)	16	1
Joint velocity	16	1
Fingertip position (4 tips, xyz)	4×3	5
Fingertip velocity	4×3	1
Object position	3	5
Object orientation (quaternion)	4	5
Object linear velocity	3	1
Object angular velocity	3	1
Fingertip contact (touch, $\log(1+\lvert f\rvert)$)	4×2	1
o^{crit} *critic*		
o^{priv}	188	–
Fingertip friction draw	4	1
Encoder-bias draw	16	1
ResSafe inputs		
a^{nom} nominal action[c]	16	1
w correction penalty	1	1
Network inputs (vector width; I enters through the depth encoder)		
π^{priv} (A1): $o^{\text{priv}}, g^{\text{priv}}$	202	
V (A1, A3): $o^{\text{crit}}, g^{\text{crit}}$	230	
π^{s} (A2, A3): o^{prop}, x, g	209	
π^{d} (A4): o^{prop}, g, I	202	
π_δ (B1): $o^{\text{prop}}, x, w, a^{\text{nom}}$	219	
V_δ (B1): $o^{\text{crit}}, w, a^{\text{nom}}$	225	
π_δ^{d} (B2): $o^{\text{prop}}, I, w, a^{\text{nom}}$	212	

[a] Limit-normalized to $[-1, 1]$ and read with the encoder bias; 10 frames = 0.5 s (privileged history: 5 frames). [b] Knife, rubber duck: position + quaternion. Lightbulb, wineglass (z-symmetric): position + goal z-axis + commanded twist rate. [c] In B1 training, perturbed by $s\boldsymbol{\epsilon}$, $s \sim \mathcal{U}(0, 0.5)$.

TABLE X: PPO hyperparameters (A1, A3, B1): rsl_rl PPO with Adam, running observation normalization and bf16. $\mathcal{M}_5 = [1024, 512, 512, 256, 256]$ hidden units, ELU.

	A1	A3	B1
Environments	2×8192^a	8192	8192
Max. iterations^b	30k	5k	4k
Wall-clock^e	15 h	2.5–2.8 h	2.5–2.6 h
Steps per env. per iteration	24	24	24
Epochs / mini-batches	5 / 4	5 / 4	5 / 4
Learning rate^c	5×10^{-4}	5×10^{-4}	5×10^{-4}
Max. gradient norm	1.0	1.0	1.0
γ / GAE λ	0.99 / 0.95	0.99 / 0.95	0.99 / 0.95
Clip / value-loss coef.	0.2 / 1.0	0.2 / 1.0	0.2 / 1.0
Entropy coefficient	0.002	0.001	0.0005
Initial action std^d	1.0	A1's	0.4
Actor / critic MLP	$\mathcal{M}_5$ / $\mathcal{M}_5$	$\mathcal{M}_5$ / $\mathcal{M}_5$	$\mathcal{M}_5$ / $\mathcal{M}_5$

[a]Two GPUs; wineglass 2×16384. [b]Deployed checkpoints (knife / lightbulb / rubber duck / wineglass): A1 28000 / 28500 / 24500 / 26500, A3 5000 / 4500 / 3500 / 5000, B1 4000. [c]Adaptive: scaled to keep the KL divergence near 0.01, within $[10^{-5}, 10^{-2}]$. [d]A learned per-joint parameter (state-independent). A3 starts from the value in the A1 checkpoint that A2 distilled (mean 0.38–0.50); its actor weights come from A2, its critic from A1. [e]Max. iterations at the median time per iteration, measured on RTX 5090 GPUs (A1 on two, A3 and B1 on one): A1 1.8–2.0 s (wineglass 3.4 s at 2×16384, 28 h), A3 1.8–2.1 s, B1 2.2–2.4 s, by object.

TABLE XI: DAgger hyperparameters (A2, A4, B2). The student executes its own actions in every rollout and regresses the teacher's mean action (MSE); AdamW (weight decay 10^{-3}, $\beta = (0.9, 0.95)$), running observation normalization, bf16. $\mathcal{M}_5 = [1024, 512, 512, 256, 256]$, $\mathcal{M}_3 = [1024, 512, 256]$ hidden units, ELU.

	A2	A4	B2
Environments	4096	512	512
Iterations^a	2000	20000	3000
Wall-clock^b	0.6 h	10 h	1.8 h
Steps per env. per iteration	24	24	24
Epochs / mini-batches	5 / 4	5 / 4	5 / 4
Learning rate	5×10^{-4}	5×10^{-4}	5×10^{-4}
Max. gradient norm	1.0	1.0	1.0
Student MLP	$\mathcal{M}_5$	$\mathcal{M}_3$ + LayerNorm	$\mathcal{M}_3$
Depth encoder	–	CNN^c	CNN^c
Aux. loss weight	–	1.0	1.0

[a]Deployed checkpoint; A2 and A4 ran without a fixed budget. [b]Iterations at the median time per iteration, measured on one RTX 5090: A2 1.0–1.1 s, A4 1.8 s, B2 2.1–2.2 s, by object. [c]Conv 32@8×8/4, 64@4×4/2, 64@3×3/1 (ReLU); spatial-softmax keypoints plus channel means (192-D), concatenated with the vector input.

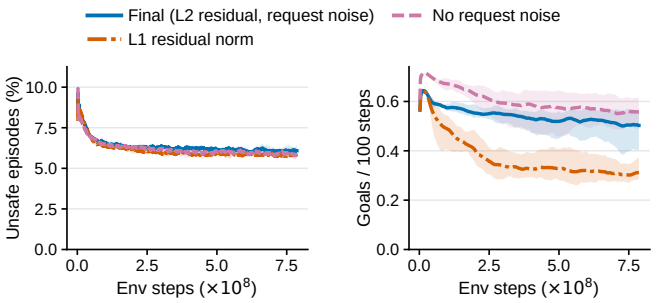


Fig. 31: **ResSafe B1 training, rubber duck.** Unsafe-episode rate (left) and goals per 100 steps (right) for the final filter (L2 penalty, nominal-action noise), an L1 penalty, and no nominal-action noise. Mean with min–max band over 3 seeds. All variants reach a similar unsafe rate, but the L1 penalty reaches far fewer goals.

TABLE XII: Training the HJ filter longer does not help (simulation, rubber duck, state policy; same protocol and columns as Table I). Iter.: training iterations. The 50k-iteration filter (about 1 h, Table I) is retrained for 180k iterations (3.6×, about 3.9 h on one GPU, ResSafe's B1 wall-clock), all else identical. Each 50k row is paired with the 180k setting closest in drop rate; the same α, β intervene about twice as much after longer training, so matching drops needs a smaller β. At matched drops the 180k filter reaches 4–9 points fewer precise goals and changes the action more. The LRSF gate has no drop-matched 180k setting: at α=0.2 it reaches 70.1 % precise goals at 7.0 % drops, against 65.5 % at 6.5 % for the 50k filter at α=0.1, so it trades a few more goals for more drops. Best of three training seeds per filter (180k: seed 2); mean ± s.e. across environments.

		Goals reached (%) ↑			Time to 1st drop or	Avg. action change (%)
Filter	Iter.	coarse	precise	Drops (%) ↓	end (s) ↑	
None	–	92.1 ±0.3	88.4 ±0.3	9.1 ±0.3	154 ±5	–
ResSafe, w=0.5	–	91.3 ±0.3	80.1 ±0.4	4.5 ±0.3	266 ±6	15.3
HCSF α=0.3, β=0.9	50k	88.2 ±0.4	79.8 ±0.5	5.9 ±0.2	199 ±6	10.3
HCSF α=0.3, β=0.75	180k	86.2 ±0.4	75.9 ±0.5	5.7 ±0.2	205 ±6	12.4
HCSF α=0.2, β=0.9	50k	83.1 ±0.5	72.1 ±0.6	4.6 ±0.1	227 ±6	15.8
HCSF α=0.2, β=0.75	180k	77.5 ±0.6	63.9 ±0.6	4.6 ±0.1	234 ±6	19.0
HCSF α=0.3, β=0.95	50k	81.6 ±0.4	69.3 ±0.5	3.9 ±0.1	253 ±6	17.3
HCSF α=0.3, β=0.9	180k	75.3 ±0.5	60.6 ±0.6	3.8 ±0.2	256 ±6	20.5

TABLE XIII: Every swept setting of the HJ filter of Table I (simulation, rubber duck, state policy; same protocol and columns). α: CVaR level; β: HCSF decay rate (larger: more conservative); LRSF switches to the fallback action when the CVaR falls below 10^{-3}. Coarse, precise: goals reached (%); Drops: % of goals; TTD: time to the first drop or end (s); Corr.: average action change (%). Means across 1,024 environments (s.e. about 0.5 for goals, 0.2 for drops, 6 s for TTD).

Gate	α	β	coarse	precise	Drops	TTD	Corr.
None			92.1	88.4	9.1	154	–
ResSafe, w=0.5			91.3	80.1	4.5	266	15.3
LRSF	0.05	–	47.6	39.6	3.2	258	38.3
LRSF	0.1	–	74.6	65.5	6.5	170	19.3
LRSF	0.2	–	85.5	79.9	7.9	167	10.0
LRSF	0.3	–	88.3	83.8	8.7	155	6.2
LRSF	0.5	–	91.5	87.4	8.9	148	2.4
HCSF	0.1	0.5	77.2	66.7	5.7	200	18.9
HCSF	0.1	0.75	74.7	62.7	4.4	230	21.3
HCSF	0.1	0.9	63.2	50.0	2.7	299	28.2
HCSF	0.1	0.95	47.9	35.3	1.5	362	36.1
HCSF	0.1	0.99	10.5	5.8	0.3	467	55.3
HCSF	0.2	0.5	87.1	81.3	7.5	167	9.1
HCSF	0.2	0.75	86.6	79.4	6.5	189	10.8
HCSF	0.2	0.9	83.1	72.1	4.6	227	15.8
HCSF	0.2	0.95	71.2	57.4	2.9	288	24.1
HCSF	0.2	0.99	17.2	10.3	0.5	444	50.6
HCSF	0.3	0.5	88.7	83.9	8.5	155	6.1
HCSF	0.3	0.75	89.4	83.9	7.8	166	6.4
HCSF	0.3	0.9	88.2	79.8	5.9	199	10.3
HCSF	0.3	0.95	81.6	69.3	3.9	253	17.3
HCSF	0.3	0.99	22.0	13.8	0.7	420	47.8

TABLE XIV: Real-robot evaluation protocols. A goal is reached if position and orientation error are both within the tolerance at any step of its window (coarse: 2 cm / 0.4 rad; precise: 1 cm / 0.2 rad; tilt only for the lightbulb and wineglass); no hold or twist condition. A drop is the object more than 25 cm from the hand for 3 consecutive motion-capture frames; the hand is then reset to a fixed grasp, the object is put back, and the run continues with the next goal. Filters run at w=0.5 unless stated.

Experiment	Protocol
Main (Tab. II)	State and depth policy, each ± filter; 4 objects × 6 wrist orientations × 50 consecutive goals, 10 s each (300 goals per object and variant; 13.3 h in total). Goals drawn with a fixed seed from a held-out set of 100k grasps per object, identical across variants. Twist command 0. Goals interrupted by a device dropout are redone; two consecutive stalled goals trigger a reset
Lightbulb twist (Tab. XVIII)	Depth policy ± filter; palm up and palm down; 50 goals of 20 s from the same held-out set, twist ±1 rad/s drawn per goal; turns counted about the bulb axis in the commanded direction while within the coarse tolerance
Wide goals (Tab. XV)	State policy ± filter; palm down; 50 goals of 10 s from a 100k-grasp set generated in a 1.5× larger box
Arm motion (Tab. XIX)	Depth policy ± filter, no motion capture; 5 s settle, then 30 s of sinusoids on the 3 UR5 wrist joints (0.15–0.27 Hz, up to ±85°); 10 trials per setting; held = object still in hand at the end
Tasks (Tab. XX)	Depth policy ± filter (w=1.5), no motion capture; hand-picked in-hand goals and scripted arm motions; 5–10 trials; stages scored from video

TABLE XV: Wide goals: goal grasps generated in a 1.5× larger canonical space than in training. State policy, palm down, one run of 50 consecutive 10 s goals per setting, without (×) and with (✓) the safety filter. Goals reached: position and orientation error both within the threshold at some step of the 10 s goal window; precise: ≤1 cm and ≤0.2 rad, coarse: ≤2 cm and ≤0.4 rad (orientation = axis tilt for the lightbulb and wineglass). Avg. action change: how much the safety filter changes the policy's action, averaged over control steps (RMS over the 16 joints of filtered minus policy action, in % of the action limit; – without the filter).

		Goals reached (/50) ↑		Drops (/50) ↓	Avg. action change (%)
Object	Safety filter	precise	coarse		
Knife	×	24	37	0	–
	✓	21	39	0	12.1
Lightbulb	×	30	42	0	–
	✓	23	38	0	11.9
Rubber duck	×	25	42	0	–
	✓	12	28	0	21.7
Wineglass	×	8	28	2	–
	✓	6	19	2	16.1

TABLE XVI: Simulated results behind Fig. 5 (1,024 envs × 50 consecutive 10 s goals). Coarse / Precise: % of goals reached within 2 cm, 0.4 rad / 1 cm, 0.2 rad; Drops: % of goals dropped; T_{drop}: mean time to first drop (s; 500 if none).

Object	Filter w	Coarse	Precise	Drops	T_{drop}
State stack (A3 + B1)					
	None	94.8	91.2	3.7	255
Knife	0.5	94.4	87.6	2.5	321
	0	94.3	85.7	2.1	330
	None	93.8	92.1	6.5	266
Lightbulb	0.5	97.2	94.9	2.4	343
	0	97.3	94.6	2.2	349
	None	92.1	88.4	9.1	154
Duck	0.5	91.3	80.1	4.5	266
	0	70.4	53.2	2.5	351
	None	88.6	67.7	6.1	287
Wineglass	0.5	91.3	64.2	2.5	325
	0	90.1	60.9	2.4	328
Depth stack (A4 + B2)					
	None	89.7	61.8	4.4	221
Knife	0.5	89.0	56.0	2.8	290
	0	88.6	55.2	2.8	281
	None	93.1	82.8	6.4	197
Lightbulb	0.5	93.6	77.9	3.2	285
	0	93.2	76.8	3.2	292
	None	87.0	57.4	10.2	148
Duck	0.5	81.7	39.1	3.4	275
	0	51.8	18.0	1.3	372
	None	84.0	50.3	4.4	238
Wineglass	0.5	83.3	46.3	3.5	273
	0	83.0	44.9	3.1	277

TABLE XVII: Object pose perceived by the depth policy: its auxiliary pose head versus motion capture, in simulation and on the real robot, replayed from the recorded wrist stereo pairs of the real state-policy runs (S, object mostly in nominal poses) and depth-policy runs (D). Median error over in-hand control steps; orientation is the full rotation for the knife and rubber duck and the axis tilt for the lightbulb and wineglass. >45°: steps with an orientation error above 45°.

	Position (mm)			Orientation (°)			>45° (%)		
		real			real			real	
Object	sim	S	D	sim	S	D	sim	S	D
Rubber duck	10.3	9.8	9.4	12.5	11.3	11.9	6	3	4
Lightbulb	10.2	9.9	10.5	6.5	4.9	6.0	1	0	1
Knife	11.0	12.0	13.7	10.1	13.1	**23.7**	5	8	**37**
Wineglass	10.1	**15.2**	**18.3**	4.4	**9.9**	**12.6**	0	**18**	**26**

TABLE XVIII: Lightbulb: reach a goal pose and spin the bulb about its axis (±1 rad/s commanded, direction drawn per goal), depth policy, one run of 50 goals (20 s each) per row. Reached: goals reached at 2 cm / 0.4 rad (bulb tilt only). Turns: rotation of the bulb about its own axis in the commanded direction, counted while the bulb is within 2 cm / 0.4 rad of the goal; $\geq\frac{1}{2}$ / ≥1: goals with at least half / one full turn; mean: mean turns per goal (fractional).

				Turns ↑		
Wrist	Safety filter	Reached ↑	Drops ↓	$\geq\frac{1}{2}$	≥1	mean
Palm up	×	**46**	0	**34**	**28**	**1.24**
	✓	43	0	25	20	0.74
Palm down	×	**44**	0	**19**	**12**	**0.63**
	✓	41	0	14	7	0.36

TABLE XIX: Holding on during a 30 s arm disturbance (Lissajous wrist motion of the UR5) while the policy tracks an in-hand goal and spins the object about its axis at the commanded rate. Trials (of 10) with the object still in hand when the motion ended. Goal height: above the palm; 15.6 / 16.5 cm are the top (99th percentile) of each object's goal range. Depth policy; filter: depth ResSafe filter (w=0.5).

			Held (/10) ↑	
	Goal height (cm)	Spin (rad/s)	No safety filter (×)	Safety filter (✓)
Lightbulb	11.5	−1.5	10/10	10/10
Lightbulb	15.6	−1.0	7/10	**10/10**
Rubber duck	13.0	0.5	8/10	**9/10**
Rubber duck	16.5	0.5	7/10	**9/10**
All			32/40	**38/40**

TABLE XX: Downstream tasks: the goal-reaching policy driven through a sequence of in-hand goals while the arm moves (cumulative stage success; a trial passes a stage only if it passed all earlier ones). Depth policy; safety filter: distilled depth ResSafe filter at w=1.5 (✓) or none (×).

Safety filter	Trials that passed each stage (cumulative) ↑		
Rubber duck: grasp, reorient & place	Grasp + lift	Reorient (base down)	Place upright
×	**8/10**	**8/10**	**7/10**
✓	7/10	6/10	3/10
Lightbulb: screw in	Upright from palm	Into socket	Screwed in (lights up)
×	**5/5**	**4/5**	**4/5**
✓	**5/5**	3/5	3/5
Wineglass: pick up & flip	Pick up	Flip 180°	Flip back
×	**10/10**	**8/10**	**7/10**
✓	**10/10**	**8/10**	6/10